Leadership Interview Answers

86 questions · one 2–3 minute spoken answer each

Tell me about yourself · management style · people · org design · delivery · technical leadership · influence · conflict · strategy · culture · failure · motivation · loop-specific questions for Amazon, Apple, Meta, Microsoft and Databricks

Prepared for Mal · October 2026 · Replace every red [VERIFY] with your real facts before use

Contents

0. Openers: introduction, style and philosophy 6

F1. Tell me about yourself 6
F2. Walk me through your career — why did you make each move? 7
F3. How would you describe your management style? 8
F4. What's your leadership philosophy? What makes a great engineering leader? 9
F5. How would your team, your peers and your manager describe you? 10
F6. What are your greatest strengths? 11
F7. What's your biggest weakness? 12
F8. How do you run effective one-on-ones? 13
F9. How do you delegate? What do you decide versus leave to your managers and engineers? 14
F10. How do you make decisions, especially with incomplete data? 15
F11. How do you measure your team's and your organization's success? 16
F12. How do you measure an engineering manager's performance, and what do you look for when hiring one? 17
F13. What would you do in your first 90 days? How would you structure and lead this team? 18

1. People management 19

B1. Tell me about a time you turned around an underperforming engineer 19
B2. Tell me about a time you had to let someone go (or managed someone out) 20
B3. How do you keep a high performer motivated and growing? 21
B4. How do you hire? Tell me about raising the hiring bar 22
B5. Tell me about a time you gave difficult feedback 23
B6. How is managing managers different from managing engineers? 24
B7. How do you support career development across your org? 25
B8. Tell me about retaining a key person who was about to leave 26

2. Org design and scaling 27

B9. Tell me about growing your organization significantly (20 → 80+) 27
B10. Tell me about a reorganization you led 28
B11. How do you lead teams across geographies and time zones? 29
B12. How do you decide on team structure (e.g., platform vs product teams)? 30
B13. What does your ideal engineering org look like in three years? (vision) 31

3. Execution and delivery 32

B14. Tell me about a project that slipped. How did you communicate it? 32
B15. How do you prioritize when everything is urgent? 33
B16. Tell me about leading through ambiguity 34
B17. Tell me about delivering under a tight deadline with roadblocks 35
B18. How do you balance technical debt against feature work? 36
B19. Tell me about a project that failed 37

4. Technical leadership 38

B20. Tell me about the most important architecture decision you made 38
B21. Tell me about a time you disagreed with a senior engineer on a technical decision 39
B22. Tell me about a time you were wrong about something technical 40
B23. How do you raise engineering standards across your organization? 41

B24. Tell me about a major incident you handled 42
B25. How do you improve the operational health of a system your team owns? 43
B26. Build vs buy — tell me about a decision 44
B27. How do you stay technical as a senior leader? 45
B28. How did you make the transition from engineer to manager? What was hardest? 46
B29. Tell me about the most technically complex thing you've worked on 47

5. Cross-functional influence 49

B30. Tell me about influencing people who didn't report to you 49
B31. Tell me about building trust with a skeptical cross-functional partner 50
B32. How do you partner with finance on infrastructure spend? 51
B33. Tell me about a disagreement with product management 52
B34. How do you communicate technical trade-offs to executives or non-technical stakeholders? 53

6. Conflict 54

B35. Tell me about a conflict between two people on your team 54
B36. Tell me about a conflict with your manager 55
B37. How do you handle a partner team that misses a commitment you depend on? 56
B38. Tell me about an unpopular decision you made 57

7. Strategy, vision and innovation 58

B39. Tell me about a multi-year technical strategy you set 58
B40. How are you driving AI adoption in your organization? 59
B41. What's the most innovative thing you've done? ("Invent and Simplify") 60
B42. Tell me about a time a customer asked for one thing but needed something else 61

8. Culture and inclusion 62

B43. How do you build a high-performing team culture? 62
B44. How do you create psychological safety? 63
B45. Tell me about building a diverse and inclusive team. 64

9. Failure, feedback and self-awareness 65

B46. What's the biggest mistake you've made as a manager? 65
B47. Tell me about critical feedback you received 66
B48. What would your team say you should improve? 67
B49. Tell me about a time you learned something new quickly (curiosity) 68

10. Motivation and fit 69

B50. Why this company? 69
B51. Why are you leaving? Why now? 70
B52. What's your favorite [company] product, and how would you improve it? 71
B53. What motivates you beyond compensation and title? Where do you want to be in five years? 72

11. Loop-specific and follow-on questions 73

F14. What's your biggest accomplishment? (Or: your most significant project) 73
F15. We see you at the level below what you applied for — why should we hire you at this level? 74
F16. What are you working on right now, what's most exciting — and what would you change about your current role? 75
F17. How do you manage up? What kind of manager do you work best with? 76
F18. Tell me about your best hire — and a hire that didn't work out. 77
F19. Tell me about a time you genuinely messed up a collaboration with another team. 78
F20. Tell me about a time budget pressure forced you to change direction mid-project. 79

F21. Our infrastructure spend has to drop 20% next quarter. What do you do? 80
F22. How do you structure on-call so it stays sustainable? 81
F23. What would you change about how [company] builds products — or AI? 82
F24. How do you balance team well-being against business needs? How do you prevent burnout? 83
F25. How do you align with stakeholders while protecting your team? 84
F26. Databricks panel Q&A — the seven questions the panel will likely ask after your presentation 85
F27. What questions do you have for us? 87

12. More reported questions 88

B54. How do you manage people at very different seniority levels — e.g., a very senior engineer and a new grad, or a principal engineer who's senior to you technically? 88
B55. Tell me about driving an initiative through to launch end to end 89
B56. Tell me about resolving a large-scale conflict — between teams or org-wide, not two individuals 90
B57. Tell me about a decision where you reasoned from first principles or where data contradicted the consensus 91
B58. Tell me about inheriting a struggling team and turning it around 92
B59. Tell me about a time you delegated something important and gave your team ownership 93

How to use these answers

Every question here has **one answer, written the way you'd say it** — about 2 to 3 minutes out loud (330–420 words). There are no timing breakdowns or labels. The structure is still there underneath: a one-line answer up front, the context, what *you* decided and why, what was hard, the result in numbers, and what you do differently now.

Red [VERIFY] text is a placeholder. Replace every one with your real fact before you say it aloud — numbers, names, dates, and the example situations themselves. If something in an answer didn't happen to you, keep the shape and tell your real story instead. Small reactions in the answers ("I was nervous", "that stung") are examples too: keep them only if they're true.

How to practice. Read an answer once, then put the page down and say it in your own words with a timer. Aim for 2:00–2:45. Don't memorize sentences — memorize the order of the story and the numbers. Record yourself once a week; you're listening for "we" where it should be "I", and for any part where you ramble.

When they cut in. Interviewers often interrupt with follow-ups. That's good — it means they're interested. The full Leadership & Behavioral Kit has the follow-up questions and answers for every one of these, plus the company chapters.

0. Openers: introduction, style and philosophy

F1. Tell me about yourself

Also asked as: “Walk me through your background.”, “Introduce yourself.”, “Give me the two-minute version of you.”

> Sure. I'm a Director of Software Engineering at The Hartford, which is roughly a senior engineering manager at most tech companies. I lead a global org of 80-plus engineers, which I grew from about 20, and we build the platforms behind a roughly $500 million specialty insurance business across Global Specialty and Small Specialty. That's the underwriting platform, our cloud modernization, intelligent document processing, and more and more applied AI.
>
> I came up as an engineer. At Accenture I became a tech lead on Marriott.com, so customer-facing systems, real traffic, and a client watching every release. Then I moved into solutions architecture and security architecture, working on mobile identity at AT&T. Identity is unforgiving. Every design choice is also a security and scale choice, and that's where I learned to think in trade-offs. Over time I kept seeing good designs fail for organizational reasons, not technical ones, so I moved into leading teams at The Hartford. That path is why I'm still comfortable in a design review or on an incident bridge.
>
> There are two things I'd point to. The first is the org itself. We quadrupled in size while delivery throughput went up roughly **[VERIFY: 3×]** and regretted attrition **[VERIFY: held steady]**. Mostly that's because I built a bench of **[VERIFY: N]** managers, and most of them were promoted from within. The second is document processing. The ask was basically "use AI to make underwriters faster." I turned that into a measurable program, with a pilot and a buy-versus-build call, and we cut submission handling time **[VERIFY: by about half]** with **[VERIFY: ~80%]** adoption.
>
> So why now? Honestly, the org is in good shape and the leaders under me are ready for more. What I want next is to do this at a bigger technical scale. I want to be somewhere engineering is the product rather than a function supporting it, where platform work, applied AI and building leaders all really matter. And that's why this role at [company] caught my attention.

F2. Walk me through your career — why did you make each move?

Also asked as: “Walk me through your resume.”, “Why did you leave each role?”, “How did you get from engineer to Director?”

> Sure. The short version is that every move widened the system I was responsible for. First it was a website, then an architecture, then a platform, and then an organization.
>
> I started as an engineer and became a tech lead at Accenture on Marriott.com **[VERIFY: dates]**. That's where I learned to ship customer-facing systems at real traffic with a client watching every release. It was great training, but I was always working on my slice of somebody else's system, and I wanted to understand things end to end. That's what pulled me out of it.
>
> So I went into solutions architecture and then security architecture, working on mobile identity at AT&T **[VERIFY: dates]**. Identity is unforgiving. Every design decision is also a security and scale decision, and that's where I learned to think in threat models and trade-offs. What surprised me in those years was how often a technically sound design died for organizational reasons: unclear ownership, the wrong team boundaries, nobody following through after the handoff. I kept feeling like the hardest problems weren't on the diagram.
>
> That's what took me to The Hartford **[VERIFY: year, and starting role — e.g., as an architect/engineering lead]**, on the specialty insurance platforms. I wanted to be inside a business owning outcomes, not advising on them. I moved into management **[VERIFY: year]** with a team of about 20 around the underwriting platform. Honestly, that shift was harder than I expected. I had to stop measuring myself by my own output and start measuring myself by the team's, and I had to unlearn the instinct to jump in and fix things myself. Looking back, I'd have taken a small people-leadership stretch earlier, while I was still an architect, so the move into management wasn't such a cliff.
>
> As the business grew I became Director **[VERIFY: year]**. Over the next few years I grew the org to 80-plus across Global Specialty and Small Specialty, with managers reporting to me. We took the underwriting platform through cloud modernization and built out intelligent document processing and applied AI for a roughly $500 million book.
>
> People sometimes ask why I stayed so long. The job kept changing under me: one team became a global org, on-prem became cloud, and manual intake became AI. So I got several careers' worth of scope in one place. The trade-off is that it's all one industry, and that's part of why I'm looking now. I want to do the same work somewhere engineering is the product and the scale is bigger. It's a pull toward something, not a push away.

F3. How would you describe your management style?

Also asked as: “What’s it like to work for you?”, “How do you run your org day to day?”, “How do you manage managers?”

I'd describe it as clear context and a high bar with real support behind it. I give feedback early and I judge us on outcomes, and all of that gets adjusted to the person in front of me.

In practice, context means every team knows the business goal, the constraints and the metric we'll be judged on, and we write down decision rights: what a team decides on its own, what goes to architecture review, and what comes to me. My managers own the how. The high bar means I'm explicit about what good looks like, with level guides, design-review expectations and SLOs. When someone falls short of it, I invest instead of just judging, with pairing, a written plan and weekly check-ins. I give feedback within days, and I ask for it back. Once a month we look at a small scorecard covering business, delivery, reliability and people.

The adapting part is easiest to show with an example. As the org grew past **[VERIFY: ~40]**, I promoted **[VERIFY: a senior engineer on the intake team]** to manage **[VERIFY: ~7]** engineers. Her team owned **[VERIFY: submission intake services]** feeding the underwriting platform, halfway through a cloud migration. On day one we sat down and wrote out her decision rights: what she decides alone, what we talk about, and what needs review. I skipped the usual "just ask me if you're unsure," because that makes new managers either timid or reckless.

In our weekly one-on-ones I asked how she'd handle things before I gave my view, and I let her make reversible calls I wouldn't have made. Honestly, the temptation was to solve her hardest problems for her, but then the team learns that I'm the real manager. Around week five I could see she was still taking the hardest tickets herself, and her engineers were waiting on her. I told her directly, and we agreed she'd stop for a month. That's hard to hear a few weeks into a new job, and I was worried it would knock her confidence.

By around **[VERIFY: day 90]** the scorecard and her team's feedback were steady, so we moved to fortnightly. Her team hit **[VERIFY: e.g., 9 of 10 sprints]** of its commitments, and she **[VERIFY: now mentors new managers]** herself. Now every new manager gets written decision rights and a 90-day plan.

What I took from it is that new managers fail at the edges they can't see. Writing those edges down and asking before telling gets people to independence faster than hovering or leaving them alone. A seasoned manager just gets the problem and more room.

F4. What’s your leadership philosophy? What makes a great engineering leader?

Also asked as: “What do you believe about leadership?”, “What separates good and great engineering leaders?”, “Who was the best leader you worked for and why?”

I think an engineering leader exists to do three things. You point the org at the problems that matter most to the business, you build the people and leaders who can solve them, and you remove the friction between them and the outcome. Everything else is overhead.

Direction, for me, mostly means being clear about what we won't do. I write the trade-offs down so that hundreds of daily decisions line up without a meeting. Building people means a leadership bench, not a hero. If my org can only make a good decision when I'm in the room, I've failed. Removing friction means mechanisms like planning, architecture review, on-call and hiring that make the right thing the easy thing. My personal test is simple: could I take **[VERIFY: three weeks]** off in a busy quarter and have the org make the decisions I'd have made, or better ones?

I learned that one the hard way. At around **[VERIFY: 50]** engineers, most cross-team technical and priority decisions were still coming to me. I was in **[VERIFY: 30+]** hours of meetings a week, decisions waited **[VERIFY: days]** for me, and my managers weren't growing because I was making their calls. It felt like being indispensable, but really I was the constraint.

So I changed three things. First, I wrote a one-page annual direction with the business, covering our top three outcomes, what we wouldn't do, and the guardrails on security, SLOs and cost. The easy answer would have been more status meetings, but that just keeps me as the router. Second, I moved technical calls to a forum instead of a person. That's an architecture review run by **[VERIFY: our senior engineers/architects]**, with a light design-doc template, where I'm a member, not the decider. I deliberately didn't name a single chief architect, because that just moves the bottleneck. Third, I gave each manager one area they fully owned, like the hiring loop, on-call or planning, and I reviewed outcomes monthly instead of approving each step.

Then I tested it. I took **[VERIFY: three weeks]** off in **[VERIFY: a busy quarter]** with a named delegate and asked for a written retro afterwards. Honestly, I was nervous that first week. The org delivered a **[VERIFY: quarter-end release on time]**. Escalations to me dropped by **[VERIFY: ~70%]**, my meeting load came down to about **[VERIFY: ~18 hours]** a week, and **[VERIFY: N]** managers are now ready for the next level.

So that's the philosophy in practice. Great leaders are measured by the leaders they leave behind, and I measure myself partly by how few decisions need me.

F5. How would your team, your peers and your manager describe you?

Also asked as: “What would your directs say about you?”, “What would your references tell me?”, “How would your peers describe working with you?”

I'll give you what I've actually heard from each group. Then I'll give you the critique they'd all add, because it's the same one.

My managers would say I'm clear and I'm in their corner. They know what matters, they own their area, and I back them publicly. One of them put it as **[VERIFY: e.g., "you tell me the truth early"] [VERIFY: ask two of your managers]**. Engineers in skip-levels would probably say I'm approachable and that I actually follow up on what they raise. My manager would say I take ambiguous problems and come back with a plan and a metric. They'd also say I'll push back when a commitment isn't realistic, but with options, not just a no **[VERIFY: quote from your last review]**.

Peers, meaning other engineering leaders and my business partners in underwriting, would say I'm reliable and I make their problem mine **[VERIFY: ask two peers]**. That one I had to earn. When we started the underwriting platform modernization, **[VERIFY: a senior underwriting leader]** told me technology had "promised and not delivered for years." His team had been burned by **[VERIFY: a failed release]**, and we needed them as testers, as champions, and for priority calls.

So rather than present the modernization plan again, which they'd heard before, I spent **[VERIFY: two days]** sitting with underwriters and asked what one thing would make their week better. I committed to fixing **[VERIFY: a slow search screen]** within **[VERIFY: four weeks]**, and I told them personally when it shipped. Then I brought underwriter champions into quarterly planning and cutover go/no-go decisions, so they shared the risk instead of just getting status reports. The real test came when **[VERIFY: a cutover slipped]**. I called that leader before the status report went out and gave him the reason and the new plan. I was worried it would undo the progress we'd made, but it did the opposite. Within about **[VERIFY: two quarters]** that group went from skeptics to our strongest advocates. We had **[VERIFY: one per line]** champion in planning and business sign-off on **[VERIFY: all]** cutovers.

Now the critique. All three groups would say I can be slow to close a contested decision when I want everyone aligned. I'll take one more meeting than I should **[VERIFY: ask them]**. That's fair. Now I set a decision date and an owner when a debate starts. It's noticeably better, though I still watch it.

I'm careful to say only what references will back up, and I'm happy to connect you with someone from each group.

F6. What are your greatest strengths?

Also asked as: “What are you best at?”, “Why should we hire you?”, “What would you bring that others wouldn’t?”

I'd say three, and I'll give you the evidence for each rather than adjectives.

The first is building organizations and leaders. I grew a global org from about 20 to 80-plus while throughput went up roughly **[VERIFY: 3×]** and regretted attrition stayed **[VERIFY: below company average]**, and **[VERIFY: N]** of my managers came up from within. I'm good at seeing what the org needs next before it breaks, whether that's the next manager, the next team boundary or the mechanism we're missing. If you asked me which strength matters most, it's this one. The other two produce one good program, and this one produces many.

The second is turning a vague business ask into a measurable program. The best example is our intelligent document processing work. Underwriters were re-keying broker documents, spending **[VERIFY: X hours]** per submission before any actual judgment, and we were losing business on turnaround across a roughly $500 million specialty book. The ask from leadership was basically "use AI to make underwriters faster." I didn't start building. I sat with underwriters and defined success as trustworthy structured data within an hour of a submission arriving, measured by handling time and by errors reaching decisions. Then, instead of asking for full funding up front, I ran a six-week pilot on one document type with **[VERIFY: ten]** underwriters and explicit exit criteria.

The third is technical judgment on platforms and risk, which comes from my architecture and security years: identity at AT&T, and Marriott.com at Accenture. It showed up in that same program. We bought standard extraction and built the specialty part ourselves, meaning the classification, the validation and the human review workflow. Honestly, my first instinct was to build our own models end to end. The vendor benchmark showed buying was better, and I changed my mind. That's the flip side of this strength. Technical judgment can turn into wanting to jump in, so I try to let the data overrule me. The same judgment is why we modernized the underwriting platform incrementally rather than rewriting it, with weekly releases and no business-stopping cutover.

On document processing, handling time went from **[VERIFY: X hours]** to **[VERIFY: Y minutes]** with **[VERIFY: ~80%]** adoption, and **[VERIFY: tech lead promoted]** off the back of it.

So my three strengths are building leaders, making ambiguity concrete, and sound platform judgment. From what I understand of this role, those are the three things it needs most.

F7. What’s your biggest weakness?

Also asked as: “What are you working on?”, “Where do you need support?”, “What’s your biggest development area as a leader?”

Honestly, my biggest one is that I can hold a contested decision open too long. When smart people disagree, my instinct is to look for alignment with one more data point or one more meeting. I keep going past the point where just deciding would serve the team better **[VERIFY: confirm this is the pattern your managers see]**.

The clearest example was during our underwriting platform modernization. Two senior engineers disagreed on **[VERIFY: whether new services should share a database or own their data]**. Both had good arguments, both teams were blocked on the answer, and it was my call. I asked for a third analysis, then another review. What finally snapped me out of it was **[VERIFY: a manager]** telling me plainly that the team had stopped caring which option won. They just wanted an answer. By then it had cost us **[VERIFY: about six weeks]**, which was longer than the downside of either option. The longer it ran, the more personal it got between the two engineers. That's the real cost: people wait, the debate gets heated, and good engineers lose momentum.

I think it comes from my architecture years, where getting it right mattered more than getting it fast, and from genuinely wanting people to buy in. Those are good instincts, but I over-apply them.

So on that decision, I called it a two-way door for the first services, chose **[VERIFY: service-owned data with a shared read replica]**, and set a review after **[VERIFY: two extractions]**. I didn't wait for the performance tests people were asking for, because they weren't going to change the answer. I also didn't just announce it in a group email. I sat down with the engineer whose option lost, explained the reasoning, and asked him to own the revisit criteria.

Since then I've changed three habits. Every contested decision gets an owner and a date when it enters architecture review. I sort decisions into one-way and two-way doors, and two-way doors get decided within **[VERIFY: a week]** with a revisit date. And I say "disagree and commit" out loud, so people know they were heard even when their view didn't win. **[VERIFY: e.g., median time to close architecture-review decisions down from ~4 weeks to ~10 days]**, and my managers tell me I'm noticeably quicker **[VERIFY: ask them]**.

It's not fixed, though. In a new place I'll naturally want more context before deciding. So I'd tell my manager about this up front and use dates and owners from week one, so it doesn't turn into an excuse.

F8. How do you run effective one-on-ones?

Also asked as: “How do you stay connected to an 80-person org?”, “What do you cover in a 1:1?”, “How do you run skip-levels?”

For me, a one-on-one is their meeting, not my status update. Status goes in writing. The one-on-one is for what's hard, what's next, and what I should be doing differently.

With my managers it's **[VERIFY: weekly, 30–45 min]**. Their agenda comes first, and then three standing items. One is people: anyone at risk or ready to grow. Another is the hardest decision they're facing. The last is what they need from me. Once a month we step back and talk about their own development instead. Every one-on-one has a shared running doc with decisions and actions, and a standing line: "what should I do differently?" I give feedback within days, privately and specifically, and I try to visibly act on what comes back.

At 80-plus people, skip-levels are how I stay connected without going around my managers. Every engineer gets one roughly **[VERIFY: twice a year, in small groups or one-on-one]**. I ask the same questions every time: what's getting in your way, what should we stop doing, and how is your manager helping you grow. Managers get the themes first, and never who said what.

The reason I'm strict about identical questions goes back to when about **[VERIFY: a third]** of the org was in **[VERIFY: India]**. Delivery metrics looked fine, but engagement there was **[VERIFY: lower]**, and one-on-ones with their managers weren't surfacing why. I could have sent a separate survey, but that gives you scores without reasons. So I ran small-group skip-levels in both locations with the same questions. That way any difference would show up in the answers rather than in how I asked. The pattern was clear. Decisions made in US-time meetings were reaching them **[VERIFY: two or three days]** later, second-hand. Honestly, that stung, because we'd thought of ourselves as one org.

My instinct was to fix it directly with the engineers, but that undermines their managers, so I took the themes to the managers first. Then we changed the mechanism. Decisions went into written logs posted the same day, we rotated planning sessions across time zones, and each team named a decision owner in both locations. **[VERIFY: A quarter later]** I went back to the same groups and told them what had changed because of what they'd said. The lag dropped to **[VERIFY: same day]**, and engagement there rose **[VERIFY: X points]**.

What I took from it is that skip-levels are only worth doing if you ask the same questions everywhere and visibly act on the answers. Otherwise they're just theatre.

F9. How do you delegate? What do you decide versus leave to your managers and engineers?

Also asked as: “What decisions do you keep?”, “How do you avoid being a bottleneck?”, “When do you step in?”

I sort decisions by how reversible they are and how big the blast radius is. Two-way doors, which covers most technical and delivery choices, get made by the team closest to the work. One-way doors like core architecture, vendor lock-in, security models and org structure get a proper review, and some of them come to me.

What I keep is a short list. That's org structure and leadership hiring, funding trade-offs across teams, the annual direction with the business, and people decisions like promotions and exits, with my managers' input. Everything else we write down. Each manager has a one-page decision-rights doc that says what they decide, what they consult on, and what needs review. For cross-team work there's always one decider, never a committee.

The example I'd use is the underwriting platform modernization. It meant **[VERIFY: a dozen-plus]** cutovers across lines of business over **[VERIFY: 18 months]**, on a roughly $500 million business. Early on, every go/no-go came to me, often late at night. Honestly, I was the slowest part of the process, and the tech leads were better informed than I was.

So I stopped approving and started writing criteria. With the tech leads and our underwriting champions, I wrote down what "go" looks like: reconciliation thresholds met, rollback tested, business sign-off, and no quarter-end window. Then I made the tech lead for each service the decider, with the underwriting champion consulted and me just informed. I didn't set up a change board, because that dilutes ownership. I did keep the real one-way doors, which were the sequence across lines of business and anything near quarter-end, because those touched business commitments. And I asked for a short retro after each cutover, so I was reviewing outcomes rather than decisions. We adjusted the criteria twice.

What worried me was the first cutover I wasn't part of. Handing that call to a tech lead feels very different on paper than it does late at night with the business waiting. But it went fine, and so did the rest. We did **[VERIFY: N]** cutovers with zero business-stopping incidents, **[VERIFY: none]** were escalated to me, and **[VERIFY: N]** of those tech leads have been promoted since.

I step in on four signals: a one-way door made without review, a business commitment at risk with no recovery plan, a people or safety issue, or a team that's stuck. When I do, I say why and when I'll step back. What I learned is that you can delegate a high-risk decision if you delegate the criteria along with it.

F10. How do you make decisions, especially with incomplete data?

Also asked as: “Tell me how you decide under uncertainty.”, “How do you make a call when the data is ambiguous?”, “How do you know when you have enough information?”

I start by writing the decision down in one sentence, along with the options and what would change my mind.

Then I look at how reversible it is and how big the blast radius is. Reversible decisions get made fast by whoever is closest to them. Irreversible ones like architecture, vendor lock-in and org structure get more data and a wider review. For most decisions I aim to decide at about 70% of the information I'd like, because waiting for 90% usually costs more than being wrong in a way you can fix. The question I ask is what a month's delay costs compared with what being wrong costs. And rather than trying to eliminate uncertainty, I protect the downside with a pilot, a staged rollout, a kill criterion, or a metric we watch.

Here's a real example. After our intelligent document processing pilot, we had good accuracy data for **[VERIFY: one document type in Small Specialty]**. Then **[VERIFY: a Global Specialty leader]** wanted it for **[VERIFY: their submissions]**, where the documents were messier and we only had **[VERIFY: a few hundred]** test samples. The cautious option was a full accuracy study first. That would have cost a quarter, and honestly, it wasn't going to change the design.

So I framed the cost of each error. A quarter's delay meant **[VERIFY: underwriter hours and lost quotes]**. Being wrong, with human review in the loop, mostly meant rework, not bad underwriting decisions. Then I made it reversible. I raised the confidence threshold so more fields went to human review, and limited it to **[VERIFY: two teams]** rather than the whole line. I agreed a kill criterion with underwriting leadership: if **[VERIFY: errors reaching decisions]** went above **[VERIFY: threshold]**, we'd pause. And I set a revisit date at **[VERIFY: six weeks]**.

The hard part was that I was asking underwriting leaders to trust the system on thinner evidence than they'd seen for the first line, and if it went badly it could set back the whole program. At the six-week review the data held up, so we lowered the threshold where accuracy proved out and expanded. We reached **[VERIFY: target]** adoption a quarter earlier than if we'd waited, errors stayed **[VERIFY: below threshold]**, and we triggered **[VERIFY: 0]** pauses.

I haven't always got this right. On that same program, my first instinct was to build our own extraction models. The benchmark showed buying was better, so I reversed it. That's why every significant decision gets a revisit date and a short write-up. With incomplete data, the job isn't to get certain. It's to make being wrong cheap and visible.

F11. How do you measure your team’s and your organization’s success?

Also asked as: “What metrics do you use?”, “How do you know your org is healthy?”, “What does your dashboard look like?”

I measure outcomes, not activity, and I keep it to one page that my leadership team reviews monthly. There are five lenses on it.

The first is business outcomes we agree with our partners each year. For us that's things like submission handling time, quote turnaround and cost to serve **[VERIFY: your real ones]**. If those don't move, nothing else matters. The second is delivery, which means the DORA metrics (deployment frequency, lead time, change failure rate and time to restore) plus how predictable we are against quarterly commitments. The third is reliability: SLO attainment on what users actually experience, incidents that reached the business, and escaped defects. The fourth is people health, which covers regretted attrition, engagement by team, internal promotions, time to productivity for new hires, and on-call load. The fifth is the user's voice: underwriter satisfaction, and whether they actually adopt what we ship.

That page didn't always exist. At about **[VERIFY: ~60]** engineers, every team reported its own metrics in its own format, and leadership reviews were mostly status. I honestly couldn't have told you whether the org was getting healthier. So I sat down with my managers and we agreed on the one page, with an owner for each metric. I deliberately didn't build a big dashboard of everything we could measure, because nobody reads those. I also insisted we compare each team to its own trend, never to each other, because the moment you rank teams on velocity, people start gaming it.

It paid off pretty quickly. Deploys on **[VERIFY: the rating-integration team's]** side were up, and on throughput alone they looked like our best team. But their change failure rate had gone from **[VERIFY: ~8%]** to **[VERIFY: ~20%]**. The manager and I dug in and found that test coverage had been cut to hit dates. I traded **[VERIFY: one sprint]** of their feature work for test automation, which meant going to the business partner and explaining why a feature was slipping for something they couldn't see. That's an uncomfortable conversation, but with the numbers on one page it was a short one.

Their change failure rate came back **[VERIFY: under 10%]**, and we had **[VERIFY: 0]** quarter-end incidents reaching underwriters. Leadership reviews now spend **[VERIFY: little]** time on status, and **[VERIFY: all]** teams are on the common scorecard.

What I took from it is that a balanced scorecard catches what a single metric hides. Throughput without quality is just borrowing from next quarter. For myself, I add two more measures: whether my managers are growing, and how few decisions need me.

F12. How do you measure an engineering manager’s performance, and what do you look for when hiring one?

Also asked as: “What makes a great EM?”, “How do you evaluate your managers?”, “Walk me through your EM interview loop.”

I judge a manager on four things. The first is results: how the team does against business outcomes. The second is people, meaning growth, retention, engagement, and whether hard calls get made on time. The third is execution: predictability, quality and reliability. The fourth is partnership, which is how product and business partners experience working with them.

To measure that, I pull evidence from several places rather than trusting one. I look at the team's scorecard trend, regretted attrition and promotions, engagement and upward feedback, skip-level themes, and what their partners tell me directly. Every new manager gets written 90-day criteria, and every manager has a development goal each half. A team that's delivering but has low engagement isn't a success. It's borrowing from the future.

I learned to hire for this the hard way. Growing from 20 to 80-plus, I needed **[VERIFY: 6–8]** managers, and our first **[VERIFY: two]** manager hires were basically interviewed like senior engineers, on technical skill. **[VERIFY: One]** of them didn't work out. That was on me, because we'd tested for the wrong job.

So I wrote a one-page manager competency guide with my peers and HR, covering those four lenses with examples at each level. The alternative was to keep hiring on technical strength plus "seems good with people," and that's how we'd got into trouble in the first place. Then I built a five-part loop that tests each piece: a people-management deep dive, an execution case, a technical design discussion with a senior engineer, an interview with an underwriting partner, and a values round. Every part has a rubric, and written feedback comes in before the debrief. After one weak hire we added a role-play of a difficult feedback conversation, and it's become one of the most telling parts of the loop.

The signals I'm looking for are evidence they've grown someone else's career, a hard people decision handled fairly, a delivery trade-off they made themselves, enough technical depth to challenge a design, and real self-awareness about a mistake. The red flags are "I" for every success and "they" for every failure, no example of hard feedback, or a team that only works when they're in the room.

The result was that **[VERIFY: N of M]** manager hires were still strong after a year, against a much weaker record before. Regretted attrition on teams under new managers was **[VERIFY: at or below org average]**, and **[VERIFY: N]** managers were promoted from within. So the lesson for me is to define the bar in writing, test every part of it, and check at day 90.

F13. What would you do in your first 90 days? How would you structure and lead this team?

Also asked as: “How do you onboard as a leader?”, “You inherit an org — what do you change first?”, “How would you set this team up?”

I'd learn before I change anything, but I'd still deliver something early, and by day 90 I'd have a written plan.

The first month is listening. I'd do one-on-ones with every manager and key engineers, plus skip-levels, my manager, and product and business partners, and ask the same questions each time: what's working, what would you change, and what should I not touch. I'd read the strategy, design docs and postmortems, and push a small change through the pipeline myself. In the second month I'd diagnose people, technology and process. Where is talent strong or stretched, what's hurting the business, and how do decisions really get made? I'd also pick one or two early wins the team has been asking for. By day 90 I'd write down what I learned, what I'll keep, my top three priorities, the metrics, and what I won't change. I'd review it with my manager, peers and managers, then start executing. And if something's on fire in week one, of course I act on it. The plan is a default, not an excuse.

That's basically what I did when I took on the **[VERIFY: ~20-person]** underwriting engineering team **[VERIFY: in year X]**. The business was growing, the backlog was **[VERIFY: months]** long, and one team was doing everything. The business honestly expected a new structure in month one, and holding off on that was uncomfortable. I held one-on-ones with every engineer and with **[VERIFY: underwriting leaders in each line]**, asking the same questions every time. Then I picked the team's top frustration, **[VERIFY: a two-day manual release process]**, and we automated it within **[VERIFY: six weeks]**. That small win bought a lot of trust.

Around **[VERIFY: ~day 90]** I wrote the plan: capability-aligned teams, a leadership bench, and a sequence for the modernization. Several people were pushing for a big-bang platform rewrite. I said no, because the risk to the business was too high and we didn't yet understand why the system looked the way it did. I split the team into the first **[VERIFY: two]** capability teams before hiring, and then grew into the structure. The business **[VERIFY: funded the plan]**, and over **[VERIFY: years]** that became 80-plus people in **[VERIFY: N]** teams without a disruptive reorg.

On how I'd structure this team, I'd hold principles and earn the specifics. Teams aligned to outcomes, clear decision rights, a real bench, and few handoffs. I'd decide the actual shape after day 60. What I won't do is reorganize, relaunch strategy or rewrite systems before I understand why things are the way they are. The early win earns the right to bigger changes; the listening tells you which ones matter.

1. People management

B1. Tell me about a time you turned around an underperforming engineer

Also asked as: “How do you handle low performers?”, “Walk me through a PIP.”

Sure. The one that comes to mind is a senior engineer who'd been one of our strongest people and then slipped for about two quarters.

He was **[VERIFY: a senior engineer on our underwriting integration team]**, and after he moved onto **[VERIFY: a new event-driven service]** he started missing sprint commitments, and his code reviews kept bouncing back with the same kinds of defects. That team supported underwriting for our roughly $500 million specialty business, and two of his peers were quietly picking up the slack. As **[VERIFY: his manager / his manager's manager]**, it was on me to get him back to the bar, or to make a fair, timely call if he couldn't get there.

So the first thing I did was figure out *why* before doing anything formal. I brought three dated examples to a one-on-one, the missed commitments and the review comments, asked what was going on, and then honestly just stopped talking. It turned out to be a skills gap on the new stack that he was embarrassed to raise. I could have gone straight to a performance plan, but that treats a skills problem like a motivation problem and it burns trust you need later. Instead, we wrote 60-day goals together, specific deliverables and a quality bar, and I told him plainly that if we didn't get there, the next step was a formal plan with HR. Then I removed the actual blocker: I paired him for two weeks with our strongest engineer on that stack, moved one deadline so he had room to learn, and switched our one-on-ones to weekly with a written recap.

The part I worried about was the team. Two people had been carrying extra load and they were frustrated. I told them directly that I'd seen it and rebalanced the work, without sharing anything about his situation. And honestly, the hard truth is I'd noticed too late. Two quarters is a long time.

He turned it around. By day 60 he was **[VERIFY: consistently meeting commitments]**, his review rework **[VERIFY: dropped by about half]**, we had **[VERIFY: no attrition]** on the team, and **[VERIFY: about 18 months later he was promoted to tech lead]**.

What I took from it is that most underperformance I've seen is a mismatch between the role and the support, not a character problem. So now anyone moving into a new technology area starts with a ramp plan and a buddy, and my managers look at rework and commitment slips monthly, so we catch it in weeks, not quarters.

B2. Tell me about a time you had to let someone go (or managed someone out)

Also asked as: “Did you ever fire someone?”, “Tell me about the hardest people decision you made.”

Yes. The hardest one was moving a manager I had promoted myself back into an engineering role.

He'd been a strong senior engineer, and I promoted him to lead **[VERIFY: a team of about eight]** while the org was growing fast. Within a year his team's attrition was **[VERIFY: about double our average]**, and my skip-levels kept coming back with the same theme: decisions changed without explanation, and people didn't feel safe raising problems. I owned whether he could succeed in the role, a fair way of finding out, and the health of that team the whole time.

So I started with specific feedback and a real chance. I shared anonymized examples from the skip-levels, and we agreed a three-month plan: an external coach, a weekly check-in with me, and two concrete behaviors, explaining decisions in writing and running retros. I also said out loud that part of this was on me, because I'd promoted him without much support. I didn't replace him right away, because that would've been unfair when I hadn't set him up to succeed.

After **[VERIFY: three months]** there was some improvement, but the core issue was still there, and then a senior engineer on his team told me she was planning to leave. That was the moment. I could have extended the plan another quarter, and honestly part of me wanted to, but it would've cost us her and a lot more trust. So I made the call. I sat down with him, with HR, and was honest: he was a valued engineer, the manager role wasn't the right fit, and I offered **[VERIFY: a senior IC role / support finding a role elsewhere]**. He chose **[VERIFY]**. He was disappointed but not surprised, because the feedback had been explicit for months.

Then I focused on the team. I explained there'd been a leadership change without any private details, covered as interim manager for **[VERIFY: six weeks]**, and hired a new manager with the team in the interview loop. Attrition came **[VERIFY: back to normal within two quarters]**, the senior engineer **[VERIFY: stayed]**, and **[VERIFY: the team met its commitments under the new manager]**.

Looking back, I waited too long because I felt responsible for promoting him. That's my real lesson. Now every new manager gets a 90-day check with criteria we agree up front, and I keep "I made a mistake promoting you" separate from "this has to change now."

B3. How do you keep a high performer motivated and growing?

Also asked as: “Tell me about developing a top performer”, “How do you retain your best engineers?”

For me it comes down to three things: find out what they actually want next, give them scope that genuinely stretches them, and make their impact visible. Praise and more work aren't the same as growth.

A good example is **[VERIFY: a senior engineer on our intelligent document processing team]**. She was our strongest technically, people already came to her for design advice, and in a career conversation she told me she wanted to lead but wasn't sure about management. IDP was a high-visibility applied-AI program for the business, so there was real opportunity there.

The first decision was the shape of the stretch. I could have just promoted her to manager, but neither of us knew if she'd enjoy it, and that's hard to undo. So instead I made her technical lead for **[VERIFY: the extraction pipeline rebuild]**, with two engineers and a cross-team dependency to manage. That tested both paths at once, technical depth and leading people. Then I coached the new skill. She tended to solve problems herself, so in our one-on-ones we'd look at who should own what. And I let her make calls I wouldn't have made, as long as they were reversible. Stepping in would have taught her I didn't trust her. Third, I made her work visible: she presented the design at our architecture review and the results to **[VERIFY: business leadership]**. And I started a written promotion case against the level guide early, not at cycle time.

Honestly, the hardest part was me. Watching her make a call I'd have made differently and keeping my mouth shut took discipline. And delegation was a real struggle for her at first; she'd rewrite other people's code, so we practiced giving feedback instead of fixing.

The project **[VERIFY: delivered X on schedule]**, she was **[VERIFY: promoted to staff engineer / engineering manager]**, and she now **[VERIFY: leads a team of N]**. We've grown **[VERIFY: N more]** leaders the same way, and I've supported **[VERIFY: two]** internal moves out of my org.

What I believe now is that high performers mostly leave when they stop growing, not over pay. So every quarter I sit down with my managers, go through a short list of our top people, and ask what the next stretch is for each one. Sometimes the honest answer is a role outside my org, and I'll support that.

B4. How do you hire? Tell me about raising the hiring bar

Also asked as: “How did you scale hiring?”, “Tell me about a hire you're proud of / a mis-hire.”

I think of hiring as a system, not a judgment call. And that mattered a lot, because we grew the org from about 20 to over 80 engineers over **[VERIFY: three to four years]** to support a roughly $500 million specialty insurance business.

The biggest risk in that kind of growth is that the bar slips quietly. The business wanted seats filled, and a fast-growing team can lower its standard without anyone deciding to. I owned the whole thing across the org: what good looks like at each level, the loop, the interviewers, and the final bar.

So the first thing I did was write competencies for each level, with examples, so interviewers were scoring against the same standard instead of gut feel. Then I built a consistent loop: a practical coding or design exercise drawn from our actual work, system design for senior roles, and a behavioral round, each with a rubric. New interviewers shadowed before they could vote, and everyone wrote feedback before the debrief, so the loudest voice in the room didn't decide. I deliberately moved away from unstructured conversations led by the hiring manager, because those drift.

The real test came when **[VERIFY: a key business deadline]** pushed us to fill **[VERIFY: six roles]** fast. The easy move was relaxing the bar just this once. I didn't. I brought in contractors for the crunch and kept the permanent bar where it was.

We also got things wrong. Our take-home was too long and we were losing strong candidates, so I cut it to a 60-minute live exercise. And we made a mis-hire, someone who interviewed well but struggled with ambiguity. When I went back through the notes, we simply hadn't tested for it, so I added a scenario question. That one stung, because it was our process, not the person.

We got to 80-plus engineers, offer acceptance went **[VERIFY: up after the live exercise]**, first-year attrition was **[VERIFY]**, **[VERIFY: N]** of those hires have since been promoted, and we went from **[VERIFY: a handful]** of interviewers to **[VERIFY: N, calibrated]**. What I've learned is the bar erodes quietly under pressure, so it can't live in one person's head, including mine. It has to live in rubrics, trained interviewers, written feedback, and a real post-mortem on every mis-hire.

B5. Tell me about a time you gave difficult feedback

Also asked as: “Delivering critical feedback someone didn't want to hear”, “How do you give direct feedback without losing morale?”

One that stands out is telling a very senior architect that he was shutting down junior engineers in design reviews.

[VERIFY: He was a very senior architect], technically excellent, but he'd dismiss proposals in a way that just ended the conversation. Over time the junior engineers stopped bringing ideas at all. And nobody had told him, partly because of his seniority. I owned giving him that feedback, getting real change, and getting an open review culture back for the org.

The first thing I did was get specific. I pulled two reviews where he'd called proposals "naive," and I knew **[VERIFY: three engineers]** had raised it in skip-levels, which I kept anonymous. General feedback like "be a bit nicer" is easy for someone like him to dismiss, so I didn't want to go in with that. I also chose to do it privately. I could have said something in the moment in a review, but that humiliates a senior person and gets you defensiveness, not change.

In the conversation I kept it simple: here's what I saw, here's the impact, people have stopped bringing ideas, and we're losing the mentoring you're best placed to give. And the ask was small and concrete: ask a question before you give your view. He pushed back, and his argument was reasonable on the surface. High standards require bluntness. The tricky part was not sounding like I was lowering the bar. So I agreed with him on the standards and separated them from the delivery. I said, "Your standards are why I want you in the room. The tone is costing us ideas." Then I asked him to write our design-review norms for the whole org, so he was part of the fix, and I sat in on later reviews to see whether it was sticking.

It worked better than I expected. Junior proposals went from **[VERIFY: near zero]** to **[VERIFY: noticeably up within two months]**, the skip-level concerns **[VERIFY: disappeared]**, and he became one of our best mentors, and the author of the review norms.

Honestly, I'd let it run longer than I should have because of who he was. What I took from it is that senior people get the least feedback, so they need it most. I give it early, privately, with specific examples, and a month later I check by asking the people who were affected, not just the person I gave it to.

B6. How is managing managers different from managing engineers?

Also asked as: “How do you build a leadership bench?”, “How do you develop first-line managers?”

The simplest way I'd put it: with engineers I shape the work directly, but with managers, their judgment becomes my output. So I lead through coaching, written decision rights and a few good signals, not by being in the work.

I learned that the hard way. As we grew from about 20 to over 80 engineers, I went from being the only manager to leading **[VERIFY: N]** managers across our specialty insurance teams. And early on I made **[VERIFY: two]** real mistakes. I stayed in far too many technical decisions, and performance ratings were inconsistent from team to team.

So the first thing I did was write down decision rights: what managers decide alone, what we decide together, and what I decide. Then I actually stepped out of the technical calls I'd been slowing down. That was hard, honestly. I came up as an engineer, and letting go of design decisions felt like losing touch. But staying hands-on had made me the bottleneck. I also had one new manager I clashed with on scope for about a quarter, before any of this was written, which was a pretty clear sign.

Second, I oversee through signals instead of presence. Weekly one-on-ones focused on each manager's biggest problem, monthly skip-levels, a short set of health metrics like predictability, on-call load and attrition risk, and design review only for one-way doors. I didn't go the route of status meetings and detailed reporting, because that reads as distrust and still misses what you hear in a skip-level.

Third, consistent systems across teams. One way to plan, review designs and run performance, with a calibration session every cycle, so an engineer gets a similar experience whoever their manager is. And I develop managers deliberately: each new one gets a 90-day plan, a mentor manager, and I'll sit in on their first hard conversations and give feedback afterwards.

Where it landed: decisions coming to me went from **[VERIFY: most technical calls]** to **[VERIFY: one-way doors and incidents only]**, ratings get calibrated every cycle, **[VERIFY: two]** of my first-line managers have been promoted, and **[VERIFY: throughput went up with quality stable]** while we scaled. So now I set decision rights before a manager even starts, and I only step into one-way-door decisions and incidents. Everything else I delegate and judge by outcome. That's really the difference.

B7. How do you support career development across your org?

Also asked as: “How will you support career development of those working under you?”, “Tell me about promoting someone.”

I think about it in three layers: individual conversations, stretch that comes from real work, and promotions that are calibrated against a clear ladder. And I measure it, because good intentions don't make it fair.

The reason I got deliberate about it: as we grew to 80-plus engineers, I noticed promotions depended too much on which manager advocated loudest. Strong engineers felt they had to become managers to grow, and our senior roles were **[VERIFY: mostly filled externally]**. That told me the system wasn't working, even if every individual manager meant well.

So the first thing I did was separate career from performance. Every engineer now has a career conversation with their manager twice a year, apart from reviews, and it ends in a growth plan with one or two stretch goals. Annual reviews alone don't work for this, because ratings crowd out the growth conversation.

Second, I tied stretch to the real work. Managers look at the next two quarters of work and match opportunities to what people said they wanted. I didn't make a training budget the main lever, because training on its own doesn't really create growth. Doing something hard with support does. Third, promotions. I introduced written cases against the ladder, with examples, calibrated across teams instead of decided manager by manager. And I sponsored **[VERIFY: a technical track up to staff level]**, so a great engineer can keep growing without having to manage.

It wasn't smooth. Some managers pushed back on written cases as overhead. And the first calibration surfaced some uncomfortable inconsistencies between teams. I had to work through those openly with my managers rather than quietly patch them, because otherwise nobody would trust the process. Over **[VERIFY: two years]** we promoted **[VERIFY: N]** engineers, internal fills for senior roles went from **[VERIFY: X%]** to **[VERIFY: Y%]**, we have **[VERIFY: an IC track to staff]**, and regretted attrition is **[VERIFY: below the company average]**.

A recent example: one senior engineer on our document processing team wanted to explore leadership, so we made her tech lead on a rebuild, started her written promotion case early, and she's now **[VERIFY: leading a team]**. What I'd do differently is put calibration and the technical track in place earlier, before we passed 50 people, not after the unfairness showed up. Development is only fair when it's a system you actually measure.

B8. Tell me about retaining a key person who was about to leave

Also asked as: “Have you lost a strong engineer?”, “How do you handle a resignation threat?”

Yes. One of my strongest engineers told me in a one-on-one that she was interviewing elsewhere, and what I found out was that the real cause was me.

She was **[VERIFY: one of my strongest staff-level engineers]** and the expert on **[VERIFY: our legacy underwriting platform]**, which supports a roughly $500 million book. Meanwhile the most interesting work, **[VERIFY: the AI and document processing initiatives]**, was going to a newer team. I owned understanding why she was looking, fixing what I could, and also reducing the business risk of all that knowledge sitting in one person's head.

So the first thing I did was not reach for money. I thanked her for telling me, and asked what was missing. She felt stuck. And when I thought about it honestly, I'd kept her on legacy because she was the expert. I'd optimized for stability over her growth, and I told her that. A counteroffer would have been the easy move, but it wouldn't have fixed being stuck.

Then I did three things. We planned a **[VERIFY: six-week]** transfer of her legacy knowledge to two other engineers, with runbooks, instead of the "once things calm down" promise, because a vague promise is exactly what had made her start interviewing. I moved her into a lead role on **[VERIFY: the AI extraction work]**. And then I corrected her pay, which was below band for her impact. But I did that last, after the real issue, not instead of it.

It wasn't free. The knowledge transfer slowed the legacy team for a few weeks, and I had to admit to her, and to myself, that I'd created the problem. She **[VERIFY: stayed and led the next phase]**. We went from one person who knew that system to **[VERIFY: three, with runbooks]**, and her role went from legacy maintainer to **[VERIFY: lead on AI extraction]**.

What I learned is that calling your best people "too valuable to move" is a risk for them and for the business. So now I do a quarterly review of knowledge concentration across the org and rotate people deliberately. And I ask every top performer each quarter what they'd want to be doing in a year, and actually act on it, so nobody has to start interviewing to get my attention.

2. Org design and scaling

B9. Tell me about growing your organization significantly (20 → 80+)

Also asked as: “How did you scale your team?”, “What broke as you grew?”

Sure. Over **[VERIFY: about four years]** I grew our engineering org from about 20 to over 80 engineers, supporting a roughly $500 million specialty insurance business across Global Specialty and Small Specialty.

The trigger was that the business was adding **[VERIFY: new products and lines]**, and my team of about 20 had become the bottleneck. We had **[VERIFY: a backlog of around X months]**, one team doing everything, across **[VERIFY: the US and India]**. I owned the structure, hiring, building managers, and keeping delivery and quality steady while we grew.

The first big decision was structure. I organized teams around business capabilities, things like **[VERIFY: submission intake, the underwriting workbench, rating and quoting integration, and a shared platform]**, so each team owned an outcome end to end. I specifically didn't organize by geography, which creates handoffs across time zones, or by front end and back end, which forces coordination on every single feature. Second, I built a leadership bench before it was urgent. I promoted **[VERIFY: three]** engineers into management and hired **[VERIFY: two]** experienced managers, each with a 90-day plan and written decision rights. Staying flat with me as the only manager just breaks around 25 people. Third, I made hiring repeatable: level-specific competencies, a structured loop with rubrics, trained interviewers and written feedback before debriefs. And then I added mechanisms that hold at 80 people, like quarterly planning with the business, architecture review for one-way doors, shared standards and a proper on-call model.

Things did break. Around 50 people, onboarding fell apart because so much knowledge lived in a few heads. We fixed it with runbooks and a structured onboarding program. And I hired managers before I'd written decision rights, so one new manager and I clashed on scope for a quarter. That was on me.

Where we ended up: 80-plus engineers with **[VERIFY: N]** managers, delivery throughput **[VERIFY: roughly tripled]**, change failure rate **[VERIFY: flat or lower]**, new hires going from **[VERIFY: about three months]** to **[VERIFY: about two weeks]** to their first production change, and regretted attrition **[VERIFY: below the company average]**, with **[VERIFY: N]** internal promotions.

What I took from it is that the constraint changes at every size. At 20 it's people, at 50 it's communication, at 80 it's consistency. So I try to change the operating model before the old one breaks, not after.

B10. Tell me about a reorganization you led

Also asked as: “Tell me about dealing with a reorg”, “How do you handle a re-org?”

The one I'd talk about is reorganizing teams across our two business lines around shared capabilities.

[VERIFY: Our Global Specialty and Small Specialty teams had grown up separately, with duplicated platforms and different practices.] Meanwhile the business strategy was moving toward **[VERIFY: shared underwriting capabilities across lines]**. So our structure just didn't match the work anymore. I owned the design, the people conversations, the communication, and protecting delivery through the change.

The first thing I did was start from outcomes, not org-chart boxes. I listed what we needed to deliver and where handoffs were slowing us down, then designed two options together with my managers. We chose one that **[VERIFY: combined the two platform teams and kept product-aligned teams per line]**. I could have designed it alone and announced it, which is faster, but then you lose the very managers who have to make it work.

Second, I told the people most affected first, individually. Two managers' scopes were changing significantly, and I had those conversations one on one before anything went out, so nobody learned about their own change in a group meeting. Then I sent one clear message: why we're doing this, what's changing, what isn't, and the timeline, followed by team meetings, open Q&A and a living FAQ.

Third, I protected delivery. We moved people at a sprint boundary and paused new initiatives for **[VERIFY: two weeks]**. There was some pressure to just do it all at once mid-sprint and get it over with, but that's how you drop commitments. What I got wrong was how long it takes people to feel settled. Some engineers felt like their old team's identity had disappeared, and two managers needed a lot more support adjusting to their new scope than I'd planned for.

On the outcomes, duplicated services went from **[VERIFY: X]** to **[VERIFY: Y]**, cross-line feature lead time got **[VERIFY: faster]**, we had **[VERIFY: zero]** regretted attrition during the change, and **[VERIFY: we met our delivery commitments]** through the transition. So the lesson for me is that a reorg isn't done when the boxes move. I now plan a full quarter of extra one-on-one time afterwards, and run a pulse survey at 30 and 90 days, because I want to measure how people are actually doing, not just whether the structure looks right.

B11. How do you lead teams across geographies and time zones?

Also asked as: “How do you manage offshore or distributed teams?”, “How do you keep a global team aligned?”

My core rule is that no team should depend on another time zone for its daily work. So each location owns complete capabilities, with local leaders who have real decision rights, a short overlap window, and async by default for everything else.

I came to that from a situation that wasn't working. As we grew to 80-plus engineers across **[VERIFY: the US and India]**, **[VERIFY: our India team was being treated as an execution arm]**. Specs came from the US, questions waited overnight, and quality and retention at that site were suffering. I owned changing how the global org worked, so every site owned outcomes.

The first decision was to give whole capabilities, not halves. I moved end-to-end ownership of **[VERIFY: the document processing service]** to the India team: design, build, and operate. The option I didn't take was keeping the design-onshore, build-offshore split. That model creates constant handoffs and, frankly, a second-class site.

Second, I made leadership local. Each site has a manager or tech lead with real decision rights, so it's not a satellite waiting for approval. Third, I set a short daily overlap window for real collaboration, and made everything else written: design docs, recorded demos, and a decision log. I avoided long overlapping meeting hours, because that just burns people out at one end. And I invested in trust: periodic travel, rotations between sites, the same rituals everywhere, and follow-the-sun on-call so nobody is routinely getting paged at 3 a.m.

The hard part was cultural. Some US engineers were reluctant to hand over ownership. And even after the change, decisions still got made in US meetings without the other site, until the decision log became a real habit. That took longer than I expected. The results: cross-site handoffs for daily work went from **[VERIFY: frequent]** to **[VERIFY: rare]**, quality at the India site **[VERIFY: improved]**, retention there **[VERIFY: improved]**, and decisions stopped waiting overnight because **[VERIFY: they're logged async]**.

What I learned is that decisions made in the "main" time zone without the other site erode trust faster than anything. So now I design the org so nobody's blocked by geography, every decision is written down, and I'd give a remote site whole ownership from day one rather than waiting for problems to force it.

B12. How do you decide on team structure (e.g., platform vs product teams)?

Also asked as: “When do you create a platform team?”, “How do you apply Conway's law?”

I start from the work and the dependencies, not the reporting lines. Teams should own an outcome end to end with as few handoffs as possible, and I only create a platform team when duplication is genuinely costly. When I do, I run it like a product.

Here's where that came from. Across Global Specialty and Small Specialty, **[VERIFY: each line had built its own versions of document intake, identity and CI/CD]**, and routine work kept needing cross-team coordination. At the same time the business was moving toward shared underwriting capabilities. I owned deciding the boundaries: what stayed with product teams and what became shared.

So first I mapped the work. Where did routine changes need coordination across teams? And I tried to line up team boundaries with the architecture boundaries we wanted, because Conway's law works both ways. What I didn't do was draw boxes around the current reporting lines, which is the tempting shortcut.

Second, I created a platform team only where duplication was costly. **[VERIFY: Identity, data pipelines and CI/CD]** went to one platform team, and product-specific capabilities stayed with the product teams. The alternative was a big central platform team that owns anything shared, and in my experience that becomes a bottleneck pretty quickly.

Third, I set it up as a product. Product teams are its customers, with SLAs, and it's measured on adoption and request lead time, not on what it builds. And I kept teams around six to nine engineers. Honestly, it didn't land perfectly. Early on the platform team built some things nobody had asked for, and one product team kept running its own pipeline. Measuring adoption and lead time, and putting product teams into the platform's planning, is what fixed that.

The results: duplicated platform capabilities went from **[VERIFY: X]** to **[VERIFY: Y]**, cross-team handoffs for routine work were **[VERIFY: reduced]**, platform request lead time is **[VERIFY: tracked and within SLA]**, and **[VERIFY: N of M]** product teams adopted the platform. The main thing I took away is that recurring coordination on routine work is the signal that your boundaries are wrong. So I treat structure as something to revisit when that signal shows up, and if I did it again I'd set the platform team's customer measures on day one.

B13. What does your ideal engineering org look like in three years? (vision)

Also asked as: “What's your vision for the org?”, “What would you change in your first year?”

In three years I'd want three things: teams that own outcomes end to end, an engineering system where shipping safely is just the default, and a leadership bench deep enough that most senior roles get filled from within. And I'd get there by learning before changing anything.

This isn't abstract for me, because I've built toward it once already. We grew our org from about 20 to over 80 engineers for a roughly $500 million specialty insurance business, alongside **[VERIFY: cloud modernization of the underwriting platform]**. When I started, **[VERIFY: releases were infrequent, teams were organized by layer, and most senior roles were hired externally]**.

On the first piece, I organized teams around business capabilities with clear APIs between them so they could move independently, rather than layer-based teams that have to coordinate on every feature. On the second, I invested in automated testing, progressive delivery and team-owned SLOs, so speed and quality stopped being a trade-off. The option I steered away from was a central release and QA gate, which feels safe but really trades speed for false safety. And on the third, career conversations, stretch roles and calibrated promotions, so we could grow our own leaders.

The honest difficulty was that getting teams to own SLOs felt like extra work at first, and progressive delivery needed real investment that competed directly with business features. I had to keep making that case. And I'll admit I wrote the first draft of the vision too much on my own before I reviewed it with the team. It didn't really stick until they'd reshaped it.

Where we got to: release frequency went from **[VERIFY: about every two months]** to **[VERIFY: weekly for modernized services]**, teams went from **[VERIFY: layer-based]** to capability-aligned, senior roles filled internally went from **[VERIFY: X%]** to **[VERIFY: Y%]**, and we're 80-plus engineers.

For a new org, I wouldn't walk in with that plan. I'd spend the first 90 days learning the architecture, the pain points and what's already strong. Then I'd write it up as a strategy doc and review it with the team before committing, and I'd measure progress every year on things like release frequency, change failure rate, handoffs per feature and internal fill rate. A vision only matters if it's expressed as mechanisms the team helped design.

3. Execution and delivery

B14. Tell me about a project that slipped. How did you communicate it?

Also asked as: “Tight deadline and you anticipate a delay?” · “Tell me about a program that slipped; what did you change?”

The one I'd pick is our underwriting platform migration, where I realized about eight weeks out that we were going to miss the date.

This was **[VERIFY: the underwriting platform migration]**, the system carrying roughly $500 million of specialty premium. About **[VERIFY: eight weeks]** before a committed date, integration testing with **[VERIFY: the rating engine]** kept surfacing data-mapping defects faster than we could fix them. The frustrating part was that the burndown still looked green. **[VERIFY: Three teams, around 25 engineers]** were committed to that date, and I owned the commitment to the business sponsor and my manager.

So the first thing I did was size it before it was certain. I spent two days with the tech leads re-estimating against integration defects instead of tasks closed, and we came out roughly **[VERIFY: five weeks]** short. I could have waited another sprint for more certainty, but that would just have removed options. Then I went to my manager and the sponsor that same week with three options: phase out **[VERIFY: two lower-volume products]**, move the date five weeks, or add contractors. I recommended phasing, with the data behind it. Contractors sounded attractive, but onboarding would have cost us more time than it saved. And from then on I sent a short written update every week, status, risks, decisions needed, so there'd be no more surprises.

Honestly, the hard part was admitting we hadn't seen it earlier. Our own metric had hidden it. The sponsor was frustrated at first, and I had to hold the line that testing was not where we'd cut. If they'd insisted on full scope, I'd have written down the risk rather than quietly shortchange testing.

We launched the main products on the original date, and the rest **[VERIFY: about four weeks later]**. **[VERIFY: There was no sustained overtime]**, and **[VERIFY: the sponsor actually asked for the weekly update format on other programs]**. On later programs we **[VERIFY: haven't had a late integration surprise of that size]**.

The real fix was the mechanism. We'd been measuring tasks, not integration. So every program I run now tracks "tested end to end" as the actual progress measure, with an integration milestone in the first third of the plan. Adding more status meetings wouldn't have helped; they measure activity, not progress. Now a slip shows up in week four, not week twenty.

B15. How do you prioritize when everything is urgent?

Also asked as: “Tell me about competing priorities” · “How do you decide what your team works on next quarter?”

I prioritize against two or three business outcomes we've agreed up front, not against whoever's shouting loudest. And I make the trade-off visible so the right person can decide with data.

A good example: in **[VERIFY: Q3 last year]**, across **[VERIFY: our underwriting teams of about 30 engineers]**, three "urgent" asks landed at once. A regulatory reporting change with a fixed deadline, a broker portal feature that sales really wanted, and a workbench performance problem underwriters were complaining about. We had capacity for roughly **[VERIFY: one and a half]** of them that quarter. I owned the sequencing recommendation, and the conversation with whoever had to wait.

So first I mapped each ask against our quarterly outcomes, which were **[VERIFY: reducing submission turnaround and launching a new specialty product]**, and looked at impact relative to effort and risk. The regulatory change went first, no debate, because the deadline and the penalty were fixed. For the other two, I got data. The performance issue was costing each underwriter about **[VERIFY: 20 minutes a day]**. The broker feature affected **[VERIFY: a small number of brokers]**. So I recommended the performance fix next, and the business leads agreed. What I didn't do was split the team across all three, which feels fair but usually means you finish nothing well.

Then I told the sales leader myself why their feature moved and when it would ship. I didn't want them hearing it secondhand. And I kept our roughly 20% reliability capacity intact, even though it was the obvious thing to raid. It wasn't painless. Sales was unhappy and escalated once, and I had to hold the decision while keeping the relationship intact. Also, my first time-lost estimate was rough, so I sampled it with **[VERIFY: a few underwriters]** to make it credible.

The regulatory change **[VERIFY: went out on time]**, the underwriter slowdown was **[VERIFY: largely eliminated]**, the broker feature **[VERIFY: shipped next quarter on the date I'd promised]**, and **[VERIFY: nothing about it escalated to my manager]**.

So the mechanism I run now is quarterly outcome-setting with business partners, a ranked backlog everyone can see, and about 20% protected capacity. Business trade-offs get decided by the business owner, with my recommendation. Technical risk calls are mine. If I did it again, I'd have sales in the room when we set the outcomes, so their requests are measured against the same yardstick from the start.

B16. Tell me about leading through ambiguity

Also asked as: “Tell me about a time the goal wasn't clear” · “Organizational ambiguity” · “A time you had to act without complete information”

> In ambiguity I try not to wait for a perfect plan. I look for the smallest experiment that answers the biggest question, and that's how a pretty vague "use AI" ask became our funded intelligent document processing program.
>
> Leadership asked us to "use AI to make underwriters faster" for our roughly $500 million specialty business. That was it. No scope, no budget line, and honestly mixed opinions about whether AI was even ready for insurance documents. I owned turning that into something concrete and measurable, or showing it wasn't worth doing.
>
> So the first thing I did was go find the real problem. I spent a week with underwriters, and the costliest thing by far was re-keying data from **[VERIFY: loss runs and applications]**, which took **[VERIFY: X hours per submission]**. The alternative would've been launching a broad "AI for underwriting" strategy with lots of use cases at once, and I think that would've spent months before we learned anything.
>
> Second, I defined success in numbers I could test. With underwriting leadership, we agreed on time saved per submission and field accuracy, plus a guardrail on errors reaching underwriting decisions. Third, I made it reversible: a six-week pilot on **[VERIFY: one document type]**, a small team, ten underwriters, and explicit exit criteria. I didn't ask for full program funding up front, because we had no evidence yet and I'd have been asking people to fund a hunch.
>
> It wasn't smooth. Privacy review turned out to be the critical path, one document type had poor scan quality, and my own first instinct to build our own models turned out to be wrong, which is a story in itself. But the pilot reached **[VERIFY: about 90%]** field accuracy and cut handling time **[VERIFY: about in half]**. Underwriters who started out **[VERIFY: skeptical] [VERIFY: asked to keep it]**. I used that data to write the business case and the governance, privacy review and who owns accuracy, and **[VERIFY: the full program was funded]**.
>
> So now I treat any ambiguous mandate the same way: find the costliest concrete problem, define success in numbers, and run the smallest reversible experiment before committing a team. And next time I'd bring privacy and compliance in during week one, and agree the guardrail before the pilot even starts.

B17. Tell me about delivering under a tight deadline with roadblocks

Also asked as: “Tell me about a time you had to deliver with limited resources” · “A time a dependency put your deadline at risk”

The one that comes to mind is a regulatory deadline where we had **[VERIFY: about ten weeks]** and two key engineers out, and we still delivered **[VERIFY: three days early]** without a weekend death march.

[VERIFY: A regulatory change required us to update rating and reporting for our specialty lines by a fixed date about ten weeks away.] Missing it meant **[VERIFY: regulatory exposure and being unable to write business in affected states]**. **[VERIFY: Two key engineers were out]**, and we depended on **[VERIFY: the enterprise data team]** for a feed change. I owned hitting that date across **[VERIFY: two teams]**, and keeping it sustainable.

The first thing I did was cut scope to the legal minimum. I sat with compliance and defined exactly what was required by the date, and wrote down what we'd do after. The default in situations like this is to keep full scope and ask for overtime, and I didn't want to do that. Tired people make mistakes, and this was a regulated flow.

Second, I went after the dependency personally. I escalated to **[VERIFY: the data team's director]** with the business impact in dollars and dates, and they prioritized our change. We could have built a workaround around their system, but that would've left fragile debt in a regulated process. Third, I split the work so the two teams could move in parallel against a shared interface we agreed in week one. And I ran a 15-minute daily risk check, just blockers and risks, with owners and same-day decisions. Not a status meeting. If something was stuck, we decided that day who'd unstick it, and I took the cross-team ones myself.

The hard part was that the data team had their own deadlines, so the escalation only worked because I came with real numbers. And losing two engineers meant we had to reassign knowledge fast, which was stressful for the people stepping in. I was honestly worried for a couple of weeks that we'd find out late about something only those two knew.

We **[VERIFY: delivered three days early]**, the deferred items **[VERIFY: shipped the following month]**, **[VERIFY: there was no sustained weekend work]**, and **[VERIFY: no material post-release defects]**. Looking back, I started the dependency conversation in week three, and it should have been week one. So that's what I do now. Any dated program, the dependency conversations start immediately, with the business impact written down. And I cut scope before I cut people's weekends.

B18. How do you balance technical debt against feature work?

Also asked as: “How do you get buy-in for tech debt?” · “When do you stop feature work to fix the platform?”

I treat tech debt as a business risk with a cost, not a moral issue. So teams get a standing chunk of capacity, and big items get argued in business terms.

The clearest example was our **[VERIFY: legacy rating integration]**. It was causing **[VERIFY: roughly a third of our incidents]**, and every product change there took **[VERIFY: about twice as long]**. The business wanted new specialty products launched, and previous asks for "cleanup time" had been turned down. So I owned getting the business to fund the fix while still delivering the roadmap.

The first thing I did was quantify it. I showed the cost of doing nothing: **[VERIFY: X engineer-weeks a quarter lost]**, the incident impact on underwriters, and how much slower product launches were in that area. I deliberately didn't ask for a feature freeze framed as engineering quality, because that's exactly what had been refused before, and it makes it sound like engineers' preference.

Second, I proposed the rebuild alongside features rather than instead of them. It was a **[VERIFY: two-quarter]** rebuild sequenced with the planned product changes in that same area, so new features landed on the new code. A big-bang rewrite with a pause would have been cleaner on paper, but too risky for a revenue system, and it would've cost us the business's trust. Third, I set up a standing allocation: each team keeps about 20% capacity for reliability and debt they choose, so small debt never needs a negotiation. And I added a simple rule that we pay down debt in the area we're already changing, which keeps the work close to value.

The hard part was making the cost of doing nothing credible. That took real work on the data. And one quarter, launch pressure tempted us to raid the reliability capacity. I held it, which wasn't popular for a few weeks. Incidents in that area came down **[VERIFY: about 70%]**, change time got **[VERIFY: back to par]**, **[VERIFY: the roadmap held]** during the rebuild, and **[VERIFY: engineer sentiment on code health improved in the survey]**.

What I learned is that debt gets funded when its cost is concrete. So I now keep a big-debt register with the cost of doing nothing for each item, and review it with business partners quarterly. Honestly, I wish I'd started that register years earlier, before it was a third of our incidents.

B19. Tell me about a project that failed

Also asked as: “A project that failed under your leadership” · “Tell me about an initiative you shut down”

The one I'd talk about is a broker self-service quoting feature. Two teams, two quarters, and almost nobody used it. And the mistake was mine.

[VERIFY: We built a self-service broker quoting feature with two teams over two quarters]. Three months after launch, usage was **[VERIFY: under 10%]** of the brokers we'd targeted. In a roughly $500 million specialty business where engineering capacity was our real constraint, that's a meaningful cost. I'd agreed to the plan and owned the delivery, so I owned the failure and what we did next.

The first thing I did was find out why, quickly. I had the team interview brokers, and the answer was humbling. They preferred emailing an underwriter they trusted, and our flow didn't handle the **[VERIFY: complex specialty risks]** they actually brought us. The tempting response was more marketing and training to push adoption. But the interviews made it clear the problem was the premise, not the polish.

Second, I owned it with leadership. In our quarterly review, I said plainly that I'd accepted an untested assumption and committed two teams without validating it. I didn't frame it as the business partner's requirements problem, even though it had been their idea. That conversation was uncomfortable. Telling the team was harder, honestly, because they'd worked really hard on it. I made a point of saying the failure was the decision, not their execution.

Third, I stopped and salvaged. I stopped the **[VERIFY: planned next phase]**, kept what was genuinely useful, **[VERIFY: the pre-fill API]**, and redirected the team to **[VERIFY: IDP]**, which went after the real bottleneck: submissions coming in by email. So the outcome is mixed, and I'm not going to dress it up. Adoption was **[VERIFY: under 10%]**, and that investment didn't pay off. But **[VERIFY: the pre-fill API got reused in IDP]**, and **[VERIFY: leadership's trust held, they later funded IDP]**.

The change that stuck is a validation gate. No initiative over **[VERIFY: one engineer-quarter]** starts without a validation step: a prototype, a concierge test, or real data. That's actually why IDP started as a six-week pilot rather than a big program. Looking back, a two-week concierge test with real brokers would have saved us two quarters. Assumptions about users are hypotheses, not requirements.

4. Technical leadership

B20. Tell me about the most important architecture decision you made

Also asked as: “A technical decision you're proud of” · “Walk me through a system your team owned.”

The most important one was choosing to modernize our underwriting platform incrementally, with a strangler approach, instead of rewriting it.

That platform carried roughly $500 million of specialty premium. It was **[VERIFY: a monolithic Java application on on-prem servers with a shared relational database]**, releasing **[VERIFY: every two months]**, with performance problems at peak season. And the architects were split: some wanted a full cloud-native rewrite, others wanted incremental change. I owned the decision and its delivery across **[VERIFY: N teams]**.

So first I framed it on explicit criteria rather than opinions: business risk during the transition, time to first value, team skills, and operating cost. We wrote it up as a decision record with the architects. A full rewrite scored badly on risk. Underwriters use this system all day, and a multi-year freeze on a revenue system wasn't acceptable. Lift-and-shift to cloud VMs was the other option, but that just moves servers without fixing coupling or release speed.

What we did was put **[VERIFY: an API gateway/routing layer]** in front of the monolith, so every channel called through it, and capabilities could move behind it without clients changing. Then we extracted by value: **[VERIFY: document intake first, then rating integration, then the workbench]**, onto **[VERIFY: the cloud platform]**. Each one had a single source of truth, **[VERIFY: CDC-based sync]** back to the monolith, reconciliation reports, and cutover line by line with a rollback path. I also gave the biggest rewrite advocate the first extracted service and had him co-write the decision record. His conviction was useful once it had a home. Service boundaries and stack choices within guardrails I delegated to the architects.

The hard parts were real. We had **[VERIFY: a cutover where reconciliation found data mismatches and we rolled back]**, and running two systems for **[VERIFY: about 18 months]** was expensive and tiring for the teams. But releases for extracted services went from **[VERIFY: every two months]** to **[VERIFY: weekly]**, peak-season incidents came **[VERIFY: down about X%]**, we had zero business-stopping cutovers, and **[VERIFY: N teams]** now own services end to end. The dual-running cost was **[VERIFY: about $X]**.

What I learned is that data migration, not code, is always the long pole. So before I approve any re-platforming now, my first question is: what's the data plan? Source of truth, sync, reconciliation, rollback. And I fund that tooling in the first quarter, not halfway through.

B21. Tell me about a time you disagreed with a senior engineer on a technical decision

Also asked as: “Disagree and commit” · “Tell me about a design review where engineers disagreed.”

Sure — the one that comes to mind is a disagreement with our principal architect about event streaming. **[VERIFY: Our principal architect wanted to adopt an event-streaming platform (e.g., Kafka) for all integrations between underwriting services.]** Honestly, I agreed with him on the long-term direction. My worry was the near term.

I owned technical direction for **[VERIFY: the underwriting platform teams]**, which carry the core workflows for a roughly $500 million specialty insurance business. Most of our integrations were simple request-response calls, and nobody on the team had ever operated a streaming platform in production. So the question wasn't whether streaming is good technology. It was whether we were ready to run it everywhere, and whether every flow actually needed it.

The first thing I did was stop us arguing about the answer and agree on how we'd judge it. I proposed three criteria: operational risk, team skills, and which use cases genuinely needed asynchronous, decoupled events. I could have just overruled him as the manager, but that's how you lose a strong engineer's commitment. And I didn't want to simply defer to him as the expert either, because the operational cost was real and somebody had to price it. So I funded a two-week spike on one high-volume flow — **[VERIFY: document processing status events]** — and we measured throughput, latency, failure handling and what on-call would look like.

The hard part was what the spike surfaced. Running streaming well means consumer lag monitoring, runbooks, a schema registry, versioning rules for when an event's shape changes. It took a while to convince him that this was real cost and not me resisting. But the data made that conversation far easier than opinions would have.

We decided together: streaming for asynchronous, high-volume flows, request-response for everything else, and operational readiness in place before any broader rollout. Then I asked him to lead the platform setup and write the standards doc, because his conviction was an asset and I wanted it pointed at making this work. We moved flows onto it — **[VERIFY: 4 over two quarters]** — with **[VERIFY: 0]** streaming-related incidents, and the simple integrations stayed as they were. The relationship came out **[VERIFY: stronger — he brought disagreements earlier]**.

What I took from it: when two smart people disagree, they're usually optimizing for different things. Now I open any contested design review by agreeing the criteria first, and if we can test it cheaply, we run a time-boxed spike instead of debating. And if the spike had gone his way, I'd have said so publicly and backed him fully.

B22. Tell me about a time you were wrong about something technical

Also asked as: “A time you were wrong” · “A time you changed your mind based on data”

Yes — the clearest one is early in our intelligent document processing program, when I pushed us to build our own extraction models, and I was wrong.

We needed to turn broker submissions into structured data for underwriters. **[VERIFY: Early in IDP, I was confident we should build custom extraction models because specialty insurance documents — loss runs, schedules, applications — vary so much.]** That reasoning wasn't crazy; specialty documents really are messy and inconsistent. But a senior engineer argued we should start with a commercial document-AI service and only customize where it fell short. I owned the build-versus-buy call, and I went with my instinct.

So we spent **[VERIFY: ~six weeks]** building a custom model and got to **[VERIFY: 82%]** field accuracy. Meanwhile she ran a quick test of the commercial service on the same labeled documents, and it hit **[VERIFY: 88%]** out of the box. My first reaction, honestly, was to look for a reason her test didn't count. So I checked it properly — same test set, same fields, a fair comparison. It held up. The option I was tempted by was more tuning on our model, and that would have been me defending sunk cost, not making a good decision.

The hard moment was standing in front of the team that had spent six weeks on that model and saying, I made the wrong call, and here's why. I was worried their work would feel wasted. It wasn't, actually — their labeled data and the specialty pieces got reused. We switched to a hybrid: commercial extraction for the standard fields, and our own models only for **[VERIFY: specialty-specific fields]** where the service struggled.

The result was field accuracy of **[VERIFY: 88%+ hybrid]**, and we launched about **[VERIFY: ~a quarter earlier]** than the custom path would have gotten us. It also freed people from undifferentiated work — **[VERIFY: X engineers freed for specialty features]**. And something I didn't expect: **[VERIFY: more engineers challenged designs openly]** afterwards, because they'd watched me change my mind in public.

What I do differently now is simple. Confidence isn't evidence. Every build-versus-buy decision in my org starts with a quick benchmark of the buy option before we commit engineers, and I deliberately ask the most skeptical person in the room to make their case — especially when I'm the one feeling sure. If I could redo it, I'd have asked for that benchmark in week one.

B23. How do you raise engineering standards across your organization?

Also asked as: “Insist on the highest standards” · “How have you improved engineering processes?”

I raise standards by making performance visible and spreading what our best team is already doing, rather than mandating a process from the top. In my experience, mandates mostly get you box-ticking.

The real example is when I took over the larger org. Quality varied a lot: **[VERIFY: some teams deployed weekly with automated tests, others monthly with manual regression and a change failure rate around 25%]**. And underwriters felt those failures directly — a bad release in a $500 million specialty business means people can't quote or bind. I owned quality and delivery across **[VERIFY: N teams and their managers]**, so it was mine to fix.

So the first thing I did was publish four DORA-style metrics for every team — deployment frequency, lead time, change failure rate and time to restore. I deliberately didn't rank teams. Ranking turns it into a blame game; just seeing the numbers side by side was enough to start the right conversations. Second, instead of importing an external framework wholesale, I sat down with our best team and wrote down what they already did: automated tests in CI, code review with a short checklist, feature flags, and a design review for one-way-door decisions. That gave the standard credibility, because it came from our own people.

Third, I piloted before scaling. I paired engineers from the strong team with a struggling team for a sprint, and once that team's change failure rate **[VERIFY: halved]**, I gave every team a quarter with capacity set aside to adopt it. That capacity matters. If you ask for a standard without giving time, you're really asking for overtime.

Honestly, I got some of it wrong early. I put in too many rules at first. One team pushed back hard on feature flags, and rather than force it, I let them propose an alternative that met the same goal. And we dropped **[VERIFY: two]** rules that added process without moving any metric.

Over roughly a year, deployment frequency went up about **[VERIFY: ~4×]** and change failure rate came down from **[VERIFY: ~25%]** to **[VERIFY: <10%]**, with **[VERIFY: all N]** teams on the standard. I still review those numbers monthly with my managers. What I learned is that standards stick when people can see the numbers and recognize the practice as their own — and when you're willing to prune anything that doesn't pay for itself. Next time I'd start with fewer rules and add only what the metrics justify.

B24. Tell me about a major incident you handled

Also asked as: “Walk me through your most severe outage” · “How do you run postmortems?”

Sure — the one I'd pick is a quarter-end outage of our underwriting workbench. On **[VERIFY: the last business day of a quarter]**, the workbench became unusable for about **[VERIFY: three hours]**. That's the busiest day of the quarter for a roughly $500 million book, and underwriters couldn't bind policies. And the part that still stings: underwriters reported it before our alerts fired.

My first decision was not to jump into the debugging myself. I came up as an engineer, so the pull was there, but if I'm in the logs, nobody's coordinating. So I took incident command: one channel, one communicator, and updates to business leaders every 30 minutes, so engineers could focus. Second, I kept the team on mitigation rather than root cause. Within **[VERIFY: 40 minutes]** we found the database connection pools were exhausted, and we **[VERIFY: scaled the service and rolled back that morning's config change]**. There was a suggestion to roll forward with a fix, but under peak load a rollback was faster and safer.

Then the postmortem. The easy story was "bad config change," and I didn't want to stop there. We ran a five-whys, and what we found was more uncomfortable: the change was safe at normal load, but **[VERIFY: a new reporting query shared the database]** under quarter-end volume. We'd never load tested at peak, and we alerted on CPU, not on what users experienced. Honestly, we had the peak traffic data and just hadn't used it. The engineer who made the change wasn't blamed — the system let it through.

So I put in mechanisms rather than just adding capacity, which would have hidden the problem. A peak-readiness load test two weeks before every quarter-end. Alerts on latency and error-rate SLOs instead of CPU. Read replicas so reporting couldn't starve the transactional path. A quarter-end change freeze. And a blameless postmortem with owned, tracked actions that I walked the business leaders through personally.

The outcome: detection now comes from SLO alerts, not from users. The postmortem actions closed **[VERIFY: all, on schedule]**, we've had **[VERIFY: 0 in 6 quarters]** comparable quarter-end incidents, and on-call pages per week are **[VERIFY: reduced]**. We also moved on-call to a primary and secondary rotation and review every page for noise.

What I took from it: our users found the outage before we did. So every SLO we set now starts from what the user experiences, and peak readiness is a scheduled event on the calendar, not an intention.

B25. How do you improve the operational health of a system your team owns?

Also asked as: “How do you reduce on-call burden?” · “How do you define and use SLOs?”

I start from what users experience, not from what the infrastructure is doing. And I treat on-call health as part of operational health — if people dread the rotation, the system isn't healthy, whatever the dashboards say.

How I actually do it: first, define SLIs and SLOs with the business — availability, latency and correctness for the workflows underwriters care about — and move alerting to SLO burn rate instead of every CPU or memory metric. Second, rank pages and incidents by cause. It's almost always a handful of causes producing most of the noise, so you fix those out of protected reliability capacity — for us, about 20% of the teams' time. Third, put mechanisms around it so it doesn't decay: a weekly operational review of SLOs and pages, a runbook for every alert, blameless postmortems with tracked actions, and game days before peak periods.

The real example is our underwriting platform after **[VERIFY: the quarter-end outage]**. On-call was noisy and reactive — **[VERIFY: ~25 pages per on-call week]**, a lot of them unactionable — and availability was **[VERIFY: below what the business expected at peak]**. I owned reliability and the rotation across **[VERIFY: N teams]**. When we ranked pages, **[VERIFY: a handful of causes]** were behind most of them. One option on the table was adding more people to the rotation to spread the load. I didn't want that — it absorbs the noise without removing a single cause.

The hard parts were human. When we agreed targets with underwriting leadership, they initially wanted 100%. I had to explain that 100% isn't a real target, and every extra nine costs something they'd rather spend on features. And some teams resisted deleting alerts they felt safer having, even though nobody acted on them. We went through them one by one and asked: what would you do if this fired at 3 a.m.? If there was no answer, it went.

Pages per on-call week went from about **[VERIFY: ~25]** to **[VERIFY: <5]**, availability reached **[VERIFY: 99.9%]**, **[VERIFY: all]** alerts now have runbooks, and on-call sentiment **[VERIFY: improved in survey]**. I chair the weekly ops review, and we track out-of-hours pages and the on-call survey there alongside the SLOs.

Looking back, I'd define SLOs with the business before the first big incident, not after. But it's the mechanisms — the weekly review, SLOs owned jointly with the business, and funded reliability time — that keep the system healthy when I'm not in the room.

B26. Build vs buy — tell me about a decision

Also asked as: “How do you decide whether to build or buy?” · “Tell me about a vendor decision”

My rule is simple: buy what's undifferentiated, build what actually makes us different, and benchmark before committing either way. The decision I'd use is our intelligent document processing program. We needed to turn broker submissions — applications, loss runs, schedules — into structured data for underwriters in a roughly $500 million specialty business. **[VERIFY: Options were building custom models, buying a commercial document-AI service, or a hybrid.]** I owned the decision and its cost and risk profile.

The first thing I did was separate commodity from differentiator. Pulling standard fields off a document — names, dates, addresses, amounts — is commodity; plenty of vendors do it well. What's ours is understanding specialty insurance documents: classifying the specialty document types, the validation rules, and the workflow an underwriter uses to review and correct. Second, I had us benchmark the commercial service and our prototype on the same labeled documents, comparing field accuracy and latency. Third, I built a three-year total cost picture — not just vendor pricing at volume, but the engineers we'd need to keep maintaining a custom model, plus lock-in, and data privacy and residency for policyholder documents. One option I turned down was a single-vendor platform that did everything end to end, including workflow. It looked simpler, but it would have locked in the exact part that's our edge, and our data with it.

And I designed for exit. We wrapped the vendor behind **[VERIFY: our own extraction interface]** with confidence scores, so we could swap providers, and we kept all the labeled correction data in-house. That data is really the asset.

The honest part: my own first instinct was to build everything, and the benchmark is what corrected me — that's a story in itself. And the privacy review of sending policyholder documents to a vendor ended up being the critical path. I should have had privacy and procurement in the room in week one.

The outcome was the hybrid. Field accuracy at launch was **[VERIFY: 88%+ hybrid]** against **[VERIFY: 82% custom]**, we launched about **[VERIFY: ~a quarter earlier]**, the three-year cost came in **[VERIFY: hybrid, $X lower]**, and we had **[VERIFY: X more]** engineers working on differentiated features instead of maintaining commodity extraction.

What I took from it: benchmark the buy option before you commit a single engineer, and always build an exit seam — your own interface plus your own data — so a buy decision never turns into a trap.

B27. How do you stay technical as a senior leader?

Also asked as: “When did you last write code?” · “How hands-on are you?”

Honestly, I haven't written production code in about five years, and I don't pretend otherwise. What I do is stay technical where my role actually needs it — on the decisions that are expensive to get wrong.

Some background: I came up as an engineer and architect. I was a tech lead on Marriott.com at Accenture and did security architecture for mobile identity at AT&T. Now I lead an org of 80-plus engineers across Global Specialty and Small Specialty, and my technical judgment matters most on a handful of big calls — things like **[VERIFY: the underwriting platform modernization]**.

So, concretely, four things. First, I read and comment on the design docs for our most important decisions, and I sit in architecture reviews for one-way doors. I don't review every design — that would make me a bottleneck for eighty people. Second, I go deep in incident reviews, **[VERIFY: all Sev1/Sev2]**. That's where you learn how systems really behave, as opposed to how the diagram says they behave. Third, I keep a personal hands-on practice: **[VERIFY: working through coding problems in Python and building a small key-value store and rate limiter to keep current.]** It's not on anyone's critical path, and I'd never claim it is, but it keeps my instincts honest. Fourth, I lean on my staff engineers for current implementation detail, and I ask them to teach me what's changed.

Where it shows up is in the calls I've engaged in deeply — **[VERIFY: e.g., strangler migration, streaming scope, IDP build vs buy]**. In the streaming debate, for example, my value wasn't knowing the framework better than our architect. It was asking about consumer lag, schema evolution and who gets paged at two in the morning. Those questions changed the decision.

The honest difficulty is time. Management load pushes technical depth out of the calendar, so I have to schedule it deliberately. And there are newer areas — **[VERIFY: e.g., LLM tooling]** — where I know the design level well but not the implementation level, and I say so when it comes up. I've also done the hands-on project in bursts, and I'd rather keep it steady.

How I'd judge whether it's working: whether my questions in design reviews actually change decisions, what engineers say in skip-levels and surveys **[VERIFY: skip-level / survey]**, and whether I can explain our systems' failure modes. At my level, technical leadership is judgment and good questions, not keystrokes.

B28. How did you make the transition from engineer to manager? What was hardest?

Also asked as: “Why did you move into management?” · “What did you have to unlearn?”

The hardest part was letting go of being the best problem-solver in the room. I had to change what I counted as success — from problems I solved to problems my team solved without me. When I moved from **[VERIFY: tech lead/architect]** into management, I was leading **[VERIFY: a team of ~8]**, and I kept doing what had made me good before. I **[VERIFY: jumped into debugging during incidents and rewrote a design my tech lead had produced]**. In my head I was helping. In reality I was the bottleneck, and I was telling the team I didn't trust them.

What turned it was a senior engineer who told me that, plainly. My first instinct was to explain why my version of the design was better — and I'm glad I didn't. I thanked him and acted on it. That's still some of the most useful feedback I've ever had.

So a few things changed. I started tracking decisions made below me and problems the team solved without me, which sounds small, but it changes what you feel good about at the end of the week. I set clear decision rights with my tech lead. And I used my engineering background differently — in design reviews and incidents I asked questions that exposed risks instead of handing over answers. I did consider staying a player-coach who owned the hardest technical work. I decided against it, because that's exactly the role that had made me the bottleneck, and I could get far more done through the team.

Honestly, it hit my identity. I was good at solving problems, and for the first few months it felt like I was doing less. During incidents I had to actively stop myself from grabbing the keyboard.

It paid off, though. Decisions escalated to me went from **[VERIFY: most]** to **[VERIFY: few]**. Delivery when I was out used to stall **[VERIFY]**, and it became **[VERIFY: unaffected]**. And the tech leads grew — **[VERIFY: N promoted]**.

Now it's built into how I develop managers. Every new manager in my org gets written decision rights, a 90-day plan, and coaching on this exact shift — I've taken **[VERIFY: N]** managers through it — because it's the most common way new managers fail. If I did it again, I'd ask for feedback on my management style in the first month, instead of waiting for someone brave enough to tell me.

B29. Tell me about the most technically complex thing you've worked on

Also asked as: “Walk me through something you designed” · “Deepest technical work you've done”

The most technically complex work I've done was **[VERIFY: the mobile identity architecture at AT&T]** — authenticating **[VERIFY: millions of]** mobile users across apps and partner services, securely, without making sign-in painful.

The problem was this. **[VERIFY: Users needed to sign in once and access many AT&T apps and partner services; the existing approach mixed per-app credentials and weak session handling.]** The stakes cut both ways. Get security wrong and you have account takeover at carrier scale. Add too much friction and users abandon sign-in or find ways around it. I was the security architect, and I owned **[VERIFY: the target security architecture and its trade-offs]**, working with engineering and partner teams.

The core design was token-based: **[VERIFY: short-lived access tokens, refresh tokens with rotation, and federation to partner services via standard protocols such as OAuth 2.0/OpenID Connect or SAML.]** In plain terms, the user signs in once and gets a short-lived access token that each app checks on every call, plus a refresh token that's used quietly in the background to get a new one. Rotating the refresh token on every use means that if one is stolen and replayed, you can spot it. Federation meant partners trusted our identity service instead of keeping their own copies of credentials. The option we moved away from was **[VERIFY: long-lived per-app sessions, which spread credentials and made revocation impossible]**.

The hardest part was one trade-off: token lifetime versus revocation speed. Short tokens mean more refresh traffic and more chances for friction. Long tokens mean that when you need to kill a session — a stolen phone, a compromised account — it stays valid too long. **[VERIFY: We chose X minutes with a revocation check at Y.]** What we rejected was **[VERIFY: checking a central session store on every request — too much latency and load]**. On top of that, I **[VERIFY: tied tokens to device identity to reduce theft and replay]**, so a token lifted off one device isn't useful on another. And I **[VERIFY: walked app and partner teams through attack paths — token theft, replay, phishing, compromised partner — and agreed mitigations.]**

The other hard part was people, not protocols. Partners had their own timelines, and security reviews can easily become the thing everyone blames for a delay — **[VERIFY: a partner integration or vulnerability you handled, and how you balanced security with partner timelines.]** It also meant the identity service sat in front of every app, so its availability mattered as much as its security.

On outcomes, I'd quote the real numbers: scale of **[VERIFY: users; auth requests per second]**, **[VERIFY: N federated]** partner integrations, sign-in success **[VERIFY]**, and security incidents in scope **[VERIFY]**. What stuck with me is that **[VERIFY: Security that adds friction gets bypassed; the best designs make the secure path the easy one.]** I still use that lens when I review designs. And if I built it today, I'd use passkeys and

> WebAuthn, OAuth 2.1 with PKCE everywhere, and continuous risk-based authentication rather than a single decision at sign-in.

5. Cross-functional influence

B30. Tell me about influencing people who didn't report to you

Also asked as: Influence without authority; led a large cross-functional project; got buy-in from a skeptical group

Sure — the clearest one is getting underwriters to adopt intelligent document processing. None of them reported to me, and several were openly skeptical. We were rolling out **[VERIFY: intelligent document processing]** across our roughly $500 million specialty business. The people whose daily work had to change were underwriters and their operations leaders in **[VERIFY: several lines]**. And their skepticism was completely rational: if extraction got a field wrong, they'd own the underwriting error, not us. I owned adoption, not just delivery, so this was my problem to solve.

The first thing I did was involve them before we built much. I asked the **[VERIFY: two most respected underwriters]** to co-design the review screen and pick which fields mattered most. The alternative was the classic build it, demo it, train people — and that gets you compliance at best. Second, I changed the headline metric. We'd been talking about automation rate, which measured our success, not their risk. I made it "errors reaching underwriting decisions," defined with them and visible on a weekly dashboard. Third, I made adoption safe: low-confidence fields were always highlighted for review, and one-click flagging of a bad extraction fed our training data, so they could see their corrections improving the system.

And I let adoption pull rather than push. I could have gone to their executives for a mandate. I didn't, because a forced rollout plus a few early errors would have killed trust for good. Teams that weren't ready joined later, and the pilot underwriters became peer champions.

It wasn't smooth. **[VERIFY: One operations leader still refused for a quarter]**, and I let his team wait rather than force a bad first experience. Scan quality on one document type also undercut trust early, and we had to be upfront about it.

We went from a pilot of **[VERIFY: 10]** underwriters to about **[VERIFY: 80%]** adoption within two quarters. Errors reaching decisions stayed **[VERIFY: below agreed threshold]**, and corrections were feeding retraining at **[VERIFY: N corrections/month]**. The most skeptical operations leader ended up as our sponsor for expanding to **[VERIFY: new document types]**.

What I learned is that people resist change when it moves risk onto them. So now I start every cross-functional rollout by asking who carries the risk if this goes wrong, and I design the metric and the workflow around that person. If they're safer, influence mostly takes care of itself.

B31. Tell me about building trust with a skeptical cross-functional partner

Also asked as: A partner who didn't trust your team; repairing a damaged relationship with the business

The one I'd share is a new head of underwriting operations who came in with a pretty low opinion of engineering — and honestly, she had her reasons.

[VERIFY: A new head of underwriting operations] joined, and her view was shaped by **[VERIFY: a history of missed dates on the underwriting platform]**. Her teams depended on our releases to plan their work and training, and her skepticism was starting to spread to **[VERIFY: other business leaders]**. In a roughly $500 million specialty business, that's not a relationship you can let drift. I owned it on the engineering side.

The first thing I did was not argue with history. I could have walked in with a deck explaining why those dates slipped — dependencies, scope changes, all true. It wouldn't have mattered. So I acknowledged the misses and asked her what "good" would look like from where she sat. Second, I made small promises and kept them: a predictable **[VERIFY: monthly]** release she could plan training around, and a written status every Friday. I wrote that status myself for the first **[VERIFY: two months]**. I deliberately avoided a big roadmap commitment to win her over fast — big promises were exactly what had broken trust in the first place.

Third, bad news early. When **[VERIFY: a release slipped a week]**, I called her the day we knew, with the reason and the new date, before she could hear it from anyone else. That was the hard moment. Calling before I had the full answer felt uncomfortable, and some of my engineers thought the Friday status was overhead. But that one call did more for the relationship than any on-time release. Once we had a track record, I invited her into quarterly planning, so we set priorities together instead of handing them over.

On-time releases went from roughly **[VERIFY: ~50%]** to **[VERIFY: ~90%+]**. Escalations from her group to my manager went from **[VERIFY: N/quarter]** to **[VERIFY: ~0]**. After **[VERIFY: two quarters]** she was co-presenting our roadmap with me, and my engineers felt the change too — **[VERIFY: engineers invited to ops reviews]**.

What I took from it: trust comes from keeping small promises and delivering bad news early. With any new partner now, I agree two or three small, verifiable commitments in the first month and a "no surprises" rule. The one thing I'd change is bringing my managers into the relationship sooner, so the trust wasn't concentrated in me.

B32. How do you partner with finance on infrastructure spend?

Also asked as: Tell me about managing a significant cost increase; a non-engineering constraint that shaped a technical decision

I think of finance as a partner whose real need is for spend to be predictable and explainable — not necessarily lower. The example is when our cloud bill outgrew our business. After we moved the underwriting platform to the cloud, monthly spend grew **[VERIFY: about 40% in two quarters]** while transaction volume grew **[VERIFY: about 15%]**. Our finance partner flagged it, and she was right to — this platform supports a roughly $500 million business. I owned the engineering response: explain it, bring it down without hurting reliability, and make it forecastable.

The first thing I did was start with transparency instead of defense. It would have been easy to say "that's the cost of modernization." Instead I sat down with her and we built a cost view by team and service, in unit terms she could reason about — **[VERIFY: cost per submission processed and cost per environment]**. That view showed three drivers: non-production environments running 24/7, over-provisioned databases, and document images sitting on expensive storage when nobody touched them after 90 days. So I put together a dated plan with dollar estimates — schedule non-prod off-hours, rightsize databases after load testing, tier old documents to cheaper storage, and reserve the steady capacity. I didn't do a flat percentage cut across teams; it's simple, but it hits reliability in random places.

Second, we changed how we forecast. Instead of extrapolating last month's bill, we forecast from business drivers — submission volume and new products times unit cost, with scenarios — and set up a monthly variance review. Third, I protected reliability explicitly. Production headroom stayed untouched, the savings came from non-prod and storage, and I put that trade-off in writing for finance.

It wasn't clean. Tagging was incomplete, so our first cost view had about **[VERIFY: ~30%]** of spend unallocated. And one rightsizing **[VERIFY: had to be rolled back after a load test]** — better to find that in a test than in production.

Run-rate came down about **[VERIFY: ~30% lower within two quarters, ~$X annualized]** while volume kept growing. Forecast variance went from **[VERIFY: ±X%]** to **[VERIFY: within ±Y%]**, with **[VERIFY: 0]** production incidents from the cost changes. And finance now brings engineering into budget planning early instead of flagging overruns after the fact.

What I do differently now: every platform gets a unit-cost metric and a driver-based forecast from day one, and teams see their own costs — with tagging standards in place before a migration, not after the bill arrives.

B33. Tell me about a disagreement with product management

Also asked as: A conflict with a product partner; when you pushed back on a launch plan

Sure — the one that comes to mind is a disagreement with our product manager about how to launch a new broker portal. **[VERIFY: Our product manager wanted to launch the new submission portal to all brokers at once for a sales event.]** The problem was that our performance tests showed degradation at **[VERIFY: about 60% of expected peak]**. Brokers are where the revenue comes from in a roughly $500 million business, so an outage in front of them, at a sales event, would have been about the worst possible moment. I owned the engineering recommendation and launch readiness.

The first thing I did was agree the goal before we argued about the plan. I asked the PM what success looked like, and we landed on a great demo at the event and no outage. That changed the conversation from "yes or no" to "how." Second, I brought data, not opinion — the load test results, and what failure would actually look like for a broker trying to submit business. I could have just refused to launch until performance was fixed, but that would have cost the sales moment and made me the obstacle. The other extreme, launching to everyone with extra on-call and hoping traffic stayed under the limit, doesn't fix capacity.

So I proposed a staged launch: **[VERIFY: Top brokers]** at the event, everyone else two weeks later after the performance fixes, with a feature flag to control exposure. My team committed to those fixes on a dated plan. And I asked the PM to present the plan to sales as theirs — because it was; we'd built it together.

The hard part was that the PM had already promised sales a full launch. So I had to help them reset expectations without losing face, which meant giving them a plan that still looked like a win, because it genuinely was one.

The sales event **[VERIFY: went well, no incidents]**, and the full launch was **[VERIFY: clean, two weeks later]**. Capacity headroom went from **[VERIFY: ~60% of peak]** to **[VERIFY: >150% of peak]** by then. And afterwards **[VERIFY: staged launches became our default]**.

What I learned: I disagree with PMs on how, rarely on what. Now I agree the goal and the success measure in writing at the start of every launch, and I run load tests earlier — so the conversation happens before any promise gets made to sales.

B34. How do you communicate technical trade-offs to executives or non-technical stakeholders?

Also asked as: Explain a complex technical decision to a business leader; how do you get funding for tech debt

Executives make decisions on money, risk and time, not architecture. So I translate the trade-off into those terms, lead with a recommendation and real options, and put it in a one-page memo that asks for a specific decision.

A real example: **[VERIFY: Our rating integration]** was causing **[VERIFY: about a third of our incidents]** and roughly doubling the time of every product change in our specialty business. But to the business executives, it sounded like "engineering wants to refactor," and earlier requests had been deferred **[VERIFY: twice]**. Those requests had gone in as technical decks and through the normal budget cycle, and neither spoke their language.

So the first thing I changed was the framing. Instead of "we need to refactor the rating integration," I said: this area causes a third of our incidents and doubles the time of every product change, and a two-quarter investment cuts that by more than half. Second, I gave three real options — do nothing, a targeted fix, or a full re-platform — each with cost, risk, time, and what we'd stop doing to fund it, and my recommendation right at the top. I didn't present only my preferred option, because executives can tell, and it takes the decision away from them. Third, I wrote it as a one-page memo and sent it before the meeting, so people could think first and we could use the meeting for questions and the decision. And I named exactly what I needed from them, and committed to report back quarterly on the same measures.

The honest difficulty was the numbers. "Doubles the time of every change" is hard to prove, and our data was **[VERIFY: partial]**. So I showed ranges and said plainly where we were confident and where we weren't. That actually helped my credibility more than false precision would have.

It got **[VERIFY: funded for two quarters]** after those two deferrals. Incidents from the rating integration came down **[VERIFY: −X%]**, lead time for a product change was **[VERIFY: roughly halved]**, and the format stuck — **[VERIFY: memo template adopted by my managers]**.

What I do now: any significant investment in my org needs a one-page memo with options and a decision ask, and I coach my managers to write them. And I try to track the cost of tech debt continuously, so the case is ready before the pain peaks rather than after.

6. Conflict

B35. Tell me about a conflict between two people on your team

Also asked as: A large-scale conflict on your team; conflict and what you learned

The one I'd pick is two senior engineers who were in constant conflict — and what I found was that the cause was structural, not personal. **[VERIFY: Two senior engineers on our platform team — one leading the API layer, one leading data migration — were in constant conflict.]** Design reviews had turned into arguments, the rest of the team was starting to take sides, and the migration **[VERIFY: was slipping]** on a platform that supports a roughly $500 million business. I needed to fix it without losing either of them; they were both strong.

The first thing I did was listen to each of them separately. The common move is a joint "clear the air" meeting, but in my experience that hardens positions, because everyone's performing for the other person. One-on-one, they both told me essentially the same story: the other one kept changing things they depended on. That was the clue. The real issue was overlapping ownership of **[VERIFY: the data contract between the API and the migration tooling]**. Neither of them clearly owned it, so each kept changing what the other relied on.

Then I brought them together with one narrow goal: agree who owns the contract, how changes get proposed, and how disagreements escalate. We added a 48-hour decision rule so nothing sat blocked for days. I could have decided ownership myself and imposed it — faster — but then they wouldn't have owned the agreement. Separately, I gave each of them direct feedback on specific things I'd seen in reviews and what I expected instead.

The hard part was that one of them **[VERIFY: initially saw the feedback as taking sides]**. I had to be really clear that the feedback was about behavior in the room, not about who was right technically. I checked in with both weekly for the next month.

Within **[VERIFY: a month]** design reviews were productive again. Blocked decisions went from **[VERIFY: days-long]** to 48 hours or less, they **[VERIFY: co-presented the migration design at architecture review]**, and **[VERIFY: both stayed]**.

What I learned is that most conflicts between good engineers are structural — unclear ownership or conflicting goals — so I look for the structure first and the personalities second. Every shared interface in my org now has a named owner and a change process, and I check for ownership overlap whenever we split work across leads.

B36. Tell me about a conflict with your manager

Also asked as: How would you disagree with your manager?; a disagreement with an executive

Sure — the clearest one is when my manager committed a launch date before engineering had estimated the work.

[VERIFY: My manager committed to the business that we'd deliver the new specialty product's rating and binding capabilities by a fixed launch date, before engineering had estimated it.] When we sized it with my leads, the honest number came out **[VERIFY: about three months longer]**. I owned delivery, and I owed him a picture he could actually act on.

My first decision was where and how to raise it. Challenging the date in a leadership meeting would have embarrassed him in front of peers and turned it into a contest of positions. So I asked for a one-on-one the next day. Second, I didn't just walk in and say "the date's impossible." I brought the scope, our capacity, and three options: launch on the date with **[VERIFY: a limited set of coverages]**, move the date, or a combination. I recommended the phased launch, clearly. Third, I told him I'd fully support whichever option he chose. The thing I wouldn't do was quietly try to hit the date with overtime — that burns the team and usually fails anyway.

The hard part was that he was frustrated. He'd made a public commitment, and I was effectively telling him it had to change. So I tried to give him a way to go back to the business with a choice to make, not a failure to report. He picked phasing, and from then on I ran phase one as if it were my own plan — because at that point, it was.

Phase one was **[VERIFY: delivered on the launch date]**, and the remaining scope came **[VERIFY: ten weeks later]**. The team avoided a death march — **[VERIFY: no forced overtime; attrition flat]**. And afterwards I proposed that engineering sizes any date before it goes to the business. He agreed, and dates committed without sizing went from **[VERIFY: common]** to **[VERIFY: none since]**.

What I learned: disagree privately, with data and options, and then commit fully. Fixing the process mattered more than winning that one argument, and honestly it was a shared learning for both of us. I teach my managers the same pattern now, and if I could change one thing, it's getting engineering into the conversation before a commitment is made — which is exactly the rule we created.

B37. How do you handle a partner team that misses a commitment you depend on?

Also asked as: Keep a partner team accountable to a dependency; a cross-team dependency that slipped

My approach is to check my own side first, then solve it directly with the other manager — share the work, shrink the scope, or decouple — and only escalate jointly if that fails. Most missed dependencies aren't bad faith. They're unclear agreements plus competing priorities.

The real example: **[VERIFY: The data platform team slipped the event feed we needed for intelligent document processing]** by **[VERIFY: several weeks]**. That put our **[VERIFY: launch for the underwriting lines]** at risk, in a roughly $500 million business. I owned our date and our relationship with that team.

So the first thing I did was look at my own side. Had the dependency actually been written down with a date and an owner? It **[VERIFY: had been agreed only verbally]**. So, honestly, part of that miss was ours. Second, I went to their manager directly, not to their director. Escalating straight up would have damaged the relationship and probably wouldn't have produced the feed any faster. In that conversation I learned they had **[VERIFY: a competing regulatory priority]** — which, in insurance, is going to win, and should.

Third, we found options together. I lent them one of my engineers, and they agreed to deliver a minimal version of the feed first. On our side we mocked the rest of the interface and integrated later. The other option was to wait for the full feed and slip our date, which would have hurt the business when there was a better path.

The cost was real, though. Lending an engineer took capacity from my own team, and **[VERIFY: my team was frustrated]** that we were doing another team's work. I had to explain why our date was worth it, and make sure that engineer's contribution was visibly recognized.

We **[VERIFY: kept]** our launch date, and their regulatory priority was **[VERIFY: also met]**. More importantly, I built a mechanism: every cross-team dependency in my programs now has a short written agreement — scope, date, owner — and a weekly check. If escalation is needed, it's joint, with both managers presenting the trade-off together. Dependencies with written agreements went from **[VERIFY: few]** to **[VERIFY: all in my programs]**, and late surprises went from **[VERIFY: N/quarter]** to **[VERIFY: fewer]**. The lesson for me was simple: write the dependency down at the start, not after the first slip. Doing that has saved more dates than any escalation ever has.

B38. Tell me about an unpopular decision you made

Also asked as: A decision your team disagreed with; a time you held a hard line

The one I'd pick is a change freeze I introduced after a quarter-end outage. Teams really didn't like it.

[VERIFY: After the quarter-end outage of our underwriting workbench], I introduced a change freeze for the **[VERIFY: last five days]** of every quarter and required load tests before peak. The teams disliked it for understandable reasons. It slowed them down exactly when the business wanted last-minute changes, and some people read it as punishment for the outage. I owned the decision and making it stick across **[VERIFY: all teams on the underwriting path]**, without burning the goodwill I'd need for everything else.

The first thing I did was explain the reasoning with data. I didn't announce it by email. I walked the teams through the incident timeline and what it cost the business, so it was clearly about risk, not blame. Second, I kept it narrow. Only production changes on the underwriting path were frozen; everything else kept moving. An org-wide freeze would have been simpler to explain and enforce, but it would have cost far more than the risk justified. Third, I built a safety valve: an exception process with a clear owner and a same-day decision, so urgent business changes weren't simply blocked. And fourth, I committed up front to reviewing it after two quarters, with data.

The hard moment came early. **[VERIFY: A business leader pushed hard for an exception in the first quarter]**. Saying no outright would have made the freeze look rigid; saying yes would have made it meaningless. I granted a narrower version of the change and documented the risk in writing. Overall, exceptions requested versus granted were **[VERIFY: N / M]**.

We've had **[VERIFY: none since]** quarter-end incidents of that kind. When we did the review, we kept the freeze but shortened it from five days to three, which the teams appreciated — it showed the review wasn't theatre. And the business changed its behavior too: instead of last-minute change requests, they now plan around the freeze.

What I learned is that an unpopular decision is much easier to accept when it's explained, narrow, and reviewed. I now put a review date on every restrictive policy I introduce. And if I did it again, I'd involve the team leads in designing the rules — that would have taken a lot of the "punishment" feeling out of it.

7. Strategy, vision and innovation

B39. Tell me about a multi-year technical strategy you set

Also asked as: Your technical vision; how do you set direction for your org

Sure — when I took on **[VERIFY: the broader specialty engineering org]**, I wrote a three-year technical strategy tied directly to where the business was going. The business strategy was to **[VERIFY: grow specialty lines and improve underwriter productivity]** across a roughly $500 million business. But the technology underneath was a set of separate legacy systems, one per line. Every new product meant touching several of them, and nothing was shared. I owned setting the strategy, getting it funded, and aligning **[VERIFY: 80+]** engineers to it.

The first thing I did, with my architects, was tie every pillar to a business outcome. We landed on three. One, incremental modernization of the underwriting platform onto **[VERIFY: cloud]**, with shared capabilities across lines instead of one system per line. Two, turning unstructured submissions into structured data — that became our intelligent document processing work. Three, an engineering system that let us release safely every week. What I deliberately didn't write was a technology-led strategy like "move everything to microservices," which sounds bold but doesn't connect to anything the business cares about. I also ruled out a big-bang re-platform as too risky, and a line-by-line approach because it would never produce shared capabilities.

Second, I made it concrete. Each pillar had yearly measures — **[VERIFY: e.g., 70% of submissions processed digitally by year two, weekly releases for all core services by year two]** — and the first two quarters of actual work spelled out. A vision document without first steps just sits on a shelf. Third, I reviewed it with business leaders, who funded it, and then walked the whole engineering org through it so every team could see where its work fit.

The hard parts: **[VERIFY: We said no to a requested new-line system build]**, which was a tough conversation. And AI moved much faster than our plan assumed. Because we review the strategy twice a year, we could respond — we **[VERIFY: expanded the IDP pillar into broader applied AI for underwriting]** rather than treating the original plan as sacred.

On results, funding moved from project-by-project to a **[VERIFY: multi-year commitment]**; digitally processed submissions are at **[VERIFY: progress vs 70% target]**; we have **[VERIFY: N]** shared capabilities across lines, and core services on weekly releases are at **[VERIFY]**.

What I've learned is that a good strategy is a set of bets you're willing to revisit. Pairing it with explicit measures and a twice-yearly review makes changing course a planned event, not a crisis. If I did it again, I'd add an explicit "what we're not doing" list from day one.

B40. How are you driving AI adoption in your organization?

Also asked as: How are you using AI in your engineering org?; AI fluency

I drive it on two tracks — AI in our product, and AI in how we build — and on both I start where the value is measurable, with a human in the loop wherever a decision carries real risk. The context: underwriters in our roughly $500 million specialty business were spending **[VERIFY: hours]** re-keying submission data, and leadership, reasonably, expected AI to move productivity. At the same time, engineering needed a responsible way to adopt **[VERIFY: AI coding assistants]**. I own the AI strategy for my org — which use cases, what guardrails, and how we measure.

On the product side, our biggest win is intelligent document processing: extraction with human review for low-confidence fields. From there we expanded into **[VERIFY: summarizing loss runs and flagging appetite mismatches]**. What I didn't do was stand up a broad AI platform before we had a proven use case — that's a common way to spend a year and show nothing. The second principle is that we keep humans in the loop and govern models like code. No fully automated coverage decisions. Underwriters see the grounding and citations behind what the model says, and model changes go through evaluation before release, the same way code goes through tests. In a regulated business, full automation for efficiency's sake isn't a trade I'm willing to make.

On the engineering side, **[VERIFY: We rolled out AI coding assistants to all engineers]** with security and review guidelines, and we measured **[VERIFY: cycle time and review outcomes before and after, plus a developer survey]** rather than assuming it helped. Across both tracks, every AI feature has an owner for accuracy, a fallback path, and a tracked cost per transaction, with privacy reviews for policyholder data. A small enablement group shares what's working across teams.

It hasn't all worked. **[VERIFY: One summarization pilot produced unreliable output]**, and we stopped it. Privacy review was the critical path more than once, and assistant adoption has been uneven across teams.

Submission handling time came down by **[VERIFY: about half]**. **[VERIFY: all]** of our AI features have an accuracy owner and a fallback, **[VERIFY: N%]** of engineers are using assistants, and we've had **[VERIFY: 0]** AI-related incidents in regulated decisions.

The main thing I've learned is to measure AI impact, not assume it. Every AI feature in my org now ships with an accuracy owner, an evaluation before release, a fallback path and a cost-per-transaction metric — and next time I'd build the evaluation set and the privacy review into week one of every project.

B41. What's the most innovative thing you've done? ("Invent and Simplify")

Also asked as: Invent and Simplify; a time you simplified something complex; a creative solution

Honestly, the most innovative thing I've done wasn't the AI model itself. It was redesigning how underwriters take in a submission, so they only look at what the model isn't sure about.

Some context. In our specialty business, about a $500M book across Global Specialty and Small Specialty, underwriters were re-keying broker submission data through **[VERIFY: a multi-step process across two systems]**. That took **[VERIFY: X hours]** per submission before anyone had made a single underwriting judgment. I owned the intelligent document processing program: the approach, the design principles and getting it delivered.

So the first decision I made was to simplify the workflow rather than automate it. The obvious path was to automate each existing step, and we'd have ended up with a faster version of the same complicated process. Instead I pushed for one review screen where only the low-confidence fields need attention. The second decision was to make confidence the design primitive. We set a confidence threshold with underwriting leadership, so the model's uncertainty decides what a human sees. That conversation was contentious, because underwriters understandably didn't want to trust a number, but agreeing it together is what made the screen work. Third, I tested cheaply before scaling. Rather than funding a full program up front, we ran a six-week pilot on one document type with ten underwriters and exit metrics we'd agreed in advance.

I'll be honest about one idea that didn't work, and it was mine. **[VERIFY: My first instinct was to build our own extraction models.]** When we benchmarked, buying extraction and building only the specialty parts was faster and better. That build-first instinct cost us **[VERIFY: a few weeks]**, and I'd rather say so than pretend the path was clean.

The results: handling time per submission dropped by **[VERIFY: about half]**, underwriters went from **[VERIFY: 2]** systems to one review screen, about **[VERIFY: Z%]** of fields were accepted without correction, and adoption went from a **[VERIFY: pilot of 10]** to **[VERIFY: ~80%]** of underwriters.

What I took from it is that the hard part of innovation is simplifying, not adding. These days I ask "what step can we remove?" before I ask "what can we automate?" And at the org level, I fund small pilots with clear exit metrics, because that's what lets teams try ideas cheaply and drop the ones that don't pan out.

B42. Tell me about a time a customer asked for one thing but needed something else

Also asked as: Customer Obsession; uncovering the real need; working backwards from the customer

Sure. The one that comes to mind is when our underwriting leaders asked for better search, and what they actually needed was to stop re-keying data. That request ended up becoming our intelligent document processing program.

[VERIFY: Underwriting leaders asked us for a faster way to search old submissions.] On paper it was straightforward. But in a ~$500M specialty business, underwriter time drives turnaround and turnaround drives revenue, so I wanted to be sure we were solving the right thing before my org put real effort into it.

So the first thing I did was have my team and me sit with underwriters and watch the work before building anything. What we saw was that they were searching old submissions mainly to re-key data that already existed somewhere else. Better search would help a little; removing the re-keying would help a lot. The option I didn't take was building the search exactly as specified. It would have shipped, it would have been fine, and we'd have missed the real problem.

The second decision was about trust. I didn't want to come back and say "you asked for the wrong thing." That's the other option I rejected, ignoring the request and pitching only the bigger idea, because it would have lost them. So we shipped a small search improvement in **[VERIFY: a few weeks]**, which showed we'd listened. Then I quantified the re-keying time, wrote a proposal for extraction as the real fix, and brought the same leaders who'd asked for search in as sponsors of the pilot, so they helped define what success looked like.

Honestly, the hard part was that conversation. Telling senior business leaders their request wasn't really the main problem, without sounding dismissive, took care. The shadowing data did most of the work. The search fix shipped in **[VERIFY: N weeks]**, and the proposal became **[VERIFY: a funded IDP program]** that later cut re-keying time per submission by **[VERIFY: about half]**. It also changed how we work: **[VERIFY: user shadowing before major features]** became a normal practice in my org.

Looking back, I'd quantify the real need even earlier, before committing to the quick win's scope. But the lesson I carry is simple. Watch the work before you build what's asked, and when you reframe a customer's request, pair it with a quick win so they know they were heard.

8. Culture and inclusion

B43. How do you build a high-performing team culture?

Also asked as: What culture do you create?; how do you build great teams

For me, high performance comes from three things together: clarity, ownership and safety. The part people underestimate is that you reinforce them with mechanisms, not slogans.

I learned that the practical way. When I grew our org from about 20 to 80+ engineers across **[VERIFY: locations]**, supporting a ~$500M specialty insurance business, the informal culture of a small team just wasn't going to survive. **[VERIFY: New managers and teams]** each brought their own norms, and I could feel it starting to fragment.

So, clarity first. Every team knows the two or three outcomes that matter this quarter and how they're measured, and we set those in quarterly planning with the business, not in engineering alone. Then ownership. Teams own outcomes end to end, including running what they build, and they make most decisions themselves within written decision rights. I did consider keeping more decisions central for consistency, but that kills ownership, and the opposite, just letting teams figure it out, fragments the culture. Written decision rights were the middle path. And safety: blameless postmortems, and honestly how I react to bad news, decide whether people can say "I don't know," "I made a mistake" or "I disagree."

The mechanisms that hold it together are public recognition of specific behaviors, a high hiring bar with structured interviews, and dealing with low performance promptly. That last one is where it got hard. **[VERIFY: One team's manager tolerated a low performer]**, and it hurt that team's morale more than any process gap did. Tolerating it to avoid conflict hurts your strongest people most, so I worked with that manager directly. Keeping the culture consistent across locations was the other genuinely hard part.

As for results, **[VERIFY: engagement scores]** moved up, regretted attrition stayed **[VERIFY: below the company average]**, delivery throughput went up **[VERIFY: about 3×]** as we grew, and we had **[VERIFY: N]** internal promotions, which tells me people could see a future in the org.

Looking back, I'd have invested in manager training on culture earlier, because at 80+ people your managers carry the culture, not you. What I believe now is that culture is what your mechanisms reinforce and what you tolerate. So I check it through data, things like engagement, attrition and how fast bad news travels, and I coach my managers on the same three things.

B44. How do you create psychological safety?

Also asked as: How do you get people to raise problems?; how do you encourage dissent

Mostly by how I react. That's the honest answer. Psychological safety is set by how leaders respond when someone brings them a problem, much more than by anything we say about it.

I noticed we had a problem as our org grew to 80+ across **[VERIFY: locations]**, in a ~$500M business. Bad news was reaching me late, **[VERIFY: a slipping project surfaced only weeks before its date]**, and our quieter engineers rarely challenged decisions in reviews. That's dangerous, because the risks you hear about late are the expensive ones.

So the first thing I changed was my own reaction. When someone tells me bad news now, the first thing out of my mouth is "thank you for telling me early." Every time. Second, I started admitting my own mistakes publicly. **[VERIFY: I shared my build-vs-buy mistake on our document processing program in an all-hands]**, including what it cost and what I changed. I'd considered keeping leadership mistakes private to protect credibility, but I think it works the other way: people don't take risks for a leader who never admits one. Third, postmortems became blameless and system-focused. After incidents, we look for the mechanism that failed, not the person. And in meetings I ask quieter people for their view first and ask "what are we missing?" Written proposals before discussion help remote and quieter engineers a lot. I didn't want to rely on an open-door policy alone, because that's passive and doesn't change how people experience leaders.

The hard moment was when **[VERIFY: one manager reacted badly to an engineer's escalation]**. It set that team back noticeably, and I had to coach him directly. It showed me that one bad reaction can undo months of work.

The results: our **[VERIFY: "I feel safe raising concerns"]** survey score went from **[VERIFY: X to Y]**, the time from a risk being known to being raised went from **[VERIFY: weeks to days]**, and we saw **[VERIFY: more challenges from junior engineers]** in design reviews, which is exactly what I wanted.

What I'd do differently is train managers on responding to bad news earlier, because day to day their reactions matter more than mine. So now I measure safety in the survey, I review that item with my managers, and I coach them on exactly that moment, the first ten seconds after someone says something's wrong.

B45. Tell me about building a diverse and inclusive team

Also asked as: How do you build diverse teams?; what have you done for inclusion

I build diverse and inclusive teams through process, because in my experience bias hides in process: who gets sourced, how we interview, and who gets the visible work.

The context was our growth from about 20 to 80+ engineers across **[VERIFY: locations]**. Early on, hiring and opportunity were pretty informal, **[VERIFY: referral-heavy sourcing and unstructured interviews]**, and our working norms suited people in one time zone with one working style. The intentions were good; the process just reproduced the team we already had.

So the first thing I did was broaden sourcing. We added **[VERIFY: partnerships with programs and community organizations]** and asked people for referrals beyond their usual networks. Referrals alone clone the existing team. Second, I structured the interview: level-specific competencies, rubrics, trained interviewers, diverse panels, and written feedback submitted before the debrief, so the loudest voice in the room doesn't set the outcome. That replaced the old unstructured "culture fit" conversations. Third, we measured the funnel, pass rates by stage, to see where candidates dropped out, and then fixed those stages, for example by **[VERIFY: rewriting job descriptions]**.

Inside the team, every quarter I review who's getting the high-visibility projects, because that's where bias quietly shows up and it compounds into promotions. We also changed how we run meetings, with written proposals before discussion and rotating meeting times, so different styles and time zones could actually contribute.

Honestly, it wasn't smooth. At our volume the funnel data was sparse, so trends took a while to show, and some of my managers saw the quarterly assignment review as overhead at first. I kept it because it was the one place I could see opportunity being distributed, not just hiring. On outcomes, we moved from unstructured interviews to structured, rubric-based loops across the org. Candidate slate diversity went from **[VERIFY]** to **[VERIFY]**, the stage pass-rate gaps **[VERIFY: narrowed]**, and representation in senior roles went from **[VERIFY: X]** to **[VERIFY: Y]**.

If I did it again, I'd start measuring the funnel and assignment distribution from the very first hire, not after the org had scaled. But the lesson stands: I don't rely on good intentions. I audit sourcing, interviewing and stretch assignments every quarter, and I hold my managers to the same standard.

9. Failure, feedback and self-awareness

B46. What's the biggest mistake you've made as a manager?

Also asked as: “Tell me about a time you genuinely messed up” (Meta's curveball); “Tell me about a time you learned from your mistakes” (Microsoft)

My biggest mistake as a manager was waiting about six months to act on a manager I'd promoted who wasn't working out. It cost us two strong engineers and some of my team's trust in me.

While I was growing the org from about 20 to 80+, I promoted **[VERIFY: a strong senior engineer]** into his first management role. Within **[VERIFY: a quarter]**, his team showed the signs: decisions changing without explanation, people afraid to raise problems, and attrition **[VERIFY: about double our average]**. As his manager, I owned his success and his team's health. My job was to see it early and act fairly, and I didn't, for **[VERIFY: about six months]**.

The first thing I had to do was be honest with myself about why. I'd promoted him, so I felt responsible, and I kept telling myself he needed more time and more coaching. I was protecting my own decision, not his team. It'd be easy to blame HR timelines or the lack of a formal process, but the delay was mine.

The moment it really landed was a skip-level where one engineer said, "we assumed you didn't know, or didn't care." That was fair, and it was hard to hear. Once I faced it, I acted directly: specific, dated feedback, a written **[VERIFY: three-month]** plan, and when that didn't work, I moved him back to an engineering role with dignity and covered the team myself while we hired with their input. I didn't want a quiet reassignment. I told the team plainly that I'd seen the signals and acted too slowly, and that it was on me, not on them.

The bigger change was turning it into mechanisms. Every new manager now gets explicit 90-day success criteria before they start, I review skip-level feedback monthly instead of quarterly, and my managers use the same approach with their new leads. Regretted attrition on that team came back **[VERIFY: to the org average within two quarters]**. The real test came later: **[VERIFY: with the next struggling manager, I acted at 90 days, he moved to a better-fit role within weeks, and we had no attrition]**.

What I learned about myself is that when I feel responsible for a situation, I'm at risk of avoiding the decision it needs. So now I treat "let's give it more time" as a signal to go look at the data, not a reason to wait.

B47. Tell me about critical feedback you received

Also asked as: “Tell me about a time you got feedback you disagreed with”; “What's the hardest feedback you've had and what did you do with it?”

> The one that stuck with me came from my own engineers: once I gave my view in a design review, the discussion stopped. They were right, and I changed how I run those reviews.
>
> This was **[VERIFY: in an upward feedback round about N years ago]**. Several engineers across my org said I made technical decisions too quickly in design reviews. And it mattered, because as we grew past **[VERIFY: 50]** engineers, those reviews were where we made the most expensive one-way-door calls on our underwriting platform.
>
> Honestly, my first reaction was defensive. I came up as an engineer, I know the platform, and I genuinely thought I was helping. So the first thing I did was not respond in the moment. I thanked people and said I'd look at it, because explaining why I weigh in early would just have proved their point.
>
> Second, I verified it rather than just accepting it or brushing it off. I deliberately watched my next **[VERIFY: few]** reviews and asked a staff engineer I trust to note when discussion stopped. It was really clear: once I spoke, alternatives stopped coming. Then I changed two specific behaviors. I speak last in design reviews now, and I open with "what would make this fail?" before I say what I think. I considered just softening my tone, or skipping the reviews, but tone doesn't remove the anchoring, and skipping removes senior judgment we actually need on one-way doors. Finally, I closed the loop publicly. I told the team what I'd heard and what I was changing, and asked them to call me out when I slipped.
>
> It wasn't smooth. In a couple of early reviews I held back so much that a weak design nearly went through, so I had to learn to ask sharper questions rather than just stay quiet. In the next feedback round, **[VERIFY: that comment was gone]**, **[VERIFY: two engineers specifically said they felt more ownership of their designs]**, and my staff engineer's notes showed **[VERIFY: more alternatives raised per review]**.
>
> What I took away is that as the most senior person in the room, my opinion carries weight whether I intend it to or not. Good questions from me create more ownership than good answers. The thing I'd do differently is ask for this kind of feedback directly and earlier, rather than waiting for a formal round to surface it.

B48. What would your team say you should improve?

Also asked as: “What's your biggest weakness as a leader?”; “What would your directs' feedback say?”

My team would say that when a project is at risk, I tend to take the recovery on myself instead of letting my managers lead it. It's real, they'd recognize it, and I'm actively working on it.

Some context. I've got 80+ engineers and **[VERIFY: N]** managers across Global Specialty and Small Specialty, so at any given time something is at risk. And my instinct, coming up from engineering, is to dive in and write the recovery plan myself. In the moment it feels responsible.

But the cost is real, and I had to name it honestly rather than call it "I just care about results," which would be a humble-brag. When I take over, my managers miss the single most developmental moment there is. The team starts seeing me as the real decision-maker. And I become a bottleneck across **[VERIFY: several]** programs at once.

So I changed my default. Now I ask my managers to bring me their recovery plan instead of writing it myself. I review it, challenge it and support it, but they own it. I did think about stepping back entirely, but that's not right either. Some risks genuinely need me in the room with the business, so I still step in when there's an escalation that needs me. Then I made it checkable. In our quarterly retro I ask my leadership team about this directly, and I track how many at-risk projects were recovered on the manager's own plan.

The hard part is the first few days of a slip. The urge to write the plan myself is strongest exactly when it matters most. **[VERIFY: One recovery took longer than I'd have done it alone]**, and sitting with that was uncomfortable. Where it stands: at-risk projects recovered on the manager's plan went from **[VERIFY: few]** to **[VERIFY: most over the last two quarters]**, and managers I'd trust to run a recovery alone went from **[VERIFY: N]** to **[VERIFY: N+]**. The retro feedback is **[VERIFY: improving, but it still comes up occasionally]**, so it's a work in progress, and I'd rather tell you that than pretend it's solved.

What I've learned is that protecting outcomes and growing leaders feel like they conflict under pressure, but they don't. The fastest durable recovery is the one my manager owns with my support. If I could redo it, I'd start this when managers are new, coaching them through their first recovery.

B49. Tell me about a time you learned something new quickly (curiosity)

Also asked as: “How do you stay current technically?”; “Tell me about something you taught yourself”

The best example is when generative AI became practical for document understanding. I taught myself enough in **[VERIFY: a few weeks]** to judge vendor claims properly, and it changed how we designed our underwriting summaries.

We already had intelligent document processing in production for underwriters on our ~$500M specialty book. Then generative AI got good enough to summarize submissions, and I started getting vendor pitches and proposals from my own team that, honestly, I couldn't evaluate well. I owned the decision on whether and how to use it in underwriting workflows, so I needed to understand it well enough to ask the right questions.

So the first thing I did was learn the fundamentals fast. I **[VERIFY: took a course on LLM fundamentals]** in the evenings, and I focused on how these models fail: hallucination, context limits, evaluation. I didn't want to rely only on vendor briefings, because vendors tell you how things work, not how they break. Second, I built something myself. I haven't written production code in about five years, but I **[VERIFY: built a small retrieval prototype over a few weekends]** on sample documents so I could see where answers drifted from the source. I could have had the team build a proof of concept, but without my own understanding I couldn't have challenged it. My prototype was pretty naive at first, which was humbling, but that's exactly where I learned where things break.

Third, I made it a team habit, not just mine. I had our ML engineers **[VERIFY: teach a weekly session]** for engineers and managers, so the whole org could reason about it.

And it changed a real decision. Because I understood why grounding answers in the source matters so much in underwriting, I insisted the summarization feature **[VERIFY: always show source citations]**, so underwriters could check every claim against the document. Citation coverage ended up at **[VERIFY: 100%]**, **[VERIFY: N]** engineers went through the teach-ins, and **[VERIFY: underwriter trust and adoption of the summaries]** followed. I also went from relying on vendor claims to asking for grounding and evaluation data on every proposal.

If I did it again, I'd bring a couple of senior engineers into the hands-on learning with me from the start. But my approach hasn't changed: I learn enough of every new technology to ask the right questions, not to be the expert, and the fastest way for me is still to build something small myself.

10. Motivation and fit

B50. Why this company?

Also asked as: “Why do you want to work here?”; “What draws you to this role?”

The short version is that I want to bring what I've learned building and scaling an engineering org to a place where technology is the product and the scale is much larger, and [company] is one of the few places where that's true for the kind of work I do.

For context, I'm Director of Software Engineering at The Hartford. I've grown a global org from about 20 to 80+ engineers across Global Specialty and Small Specialty, which is around a $500M business. We modernized the underwriting platform, moved it to the cloud incrementally, and put intelligent document processing and applied AI into production for underwriters. It's work I'm proud of, and the team's in a strong place.

When I started thinking about what's next, I was deliberate about it rather than applying broadly. I looked for four things: a genuine personal connection to the product or mission, scale beyond what I have today, a role where my platform, AI and org-building experience is useful on day one, and a culture of mechanisms and written thinking that matches how I already lead. I seriously considered staying and growing my scope internally. It's a good company, and that would have been comfortable, but I'd be optimizing what exists rather than learning at a new scale. I also passed on lateral moves at peer insurers, because they'd use the same muscles I already have.

For [company] specifically, the connection for me is **[VERIFY: your specific product/ moment]**. That's what made me want this role rather than just a role. And the fit is pretty direct. I've scaled engineering orgs and led leaders, I've run platform modernization where reliability really mattered, I've put AI into production in a regulated environment, and my earlier security architecture work on AT&T mobile identity means I bring real privacy and security rigor to product decisions.

Honestly, the hard part is leaving a team I built, and I'll be learning a new domain. But I've done that before. I moved from security architecture into insurance and had to learn the business from the ground up.

What I'd want to learn here is **[VERIFY: the specific thing you want to learn or build here]**. The way I think about it, the right next company is one where my experience is genuinely useful on day one, and where I'll still be learning in year three. [company] is that for me.

B51. Why are you leaving? Why now?

Also asked as: “Why are you looking?”; “What would make you stay?”

Honestly, I'm not leaving to get away from anything. I'm proud of what we've built at The Hartford and the team is in a strong position. I'm moving because I'm pulled toward larger scale and harder technical problems.

Over the last several years I grew the org from about 20 to 80+ engineers, led the underwriting platform modernization and cloud migration, and put intelligent document processing into production for underwriters across a ~$500M specialty business. The team now has **[VERIFY: a strong leadership bench]**, managers who could step up and run things without me. What I'm looking for next is **[VERIFY: larger scale and harder technical problems / to work where technology is the product / to build data and AI platforms rather than consume them]**.

On why now, that was a deliberate choice. I timed it around the team, not around me. I didn't want to leave in the middle of the modernization, so I waited until the major programs were stable and my managers were ready to lead without me. Leaving shouldn't put the business at risk, and I'm committed to a clean transition.

I also considered staying, seriously. There's more I could do there, it's a good company, and growing my scope internally was a real option. But the next step for my own growth is scale, and working where technology is the product. I've also been selective. I'm only talking to a few companies where the role genuinely fits, not chasing any senior title that comes up, and a lateral move within financial services wouldn't give me what I'm after.

The hard part, if I'm being straight with you, is that leaving a team you built is genuinely difficult. A lot of those people are folks I hired or promoted, and I care how they do after I'm gone. That's actually how I'll judge whether I did my job: whether the org keeps running well once I've left. So the way I see it, the org is at 80+ engineers with a real leadership bench, the platform is modernized and in the cloud, and AI is in production. The next chapter for me is leading a larger org of leaders, on platforms at much bigger scale, with AI as a product capability. [company] is one of very few places where I can do that.

If anything, I might have explored this kind of role **[VERIFY: a little earlier]**. But I think the best time to move is when what you built can run without you, and that's also the best evidence you're ready for more.

B52. What's your favorite [company] product, and how would you improve it?

Also asked as: “Which of our products do you use, and what would you change?”; “Critique one of our products”

The one I'd pick is **[VERIFY: your specific product/moment]**. I chose it because I genuinely use it, so I can talk about it from my own experience rather than from reviews. I'd rather do that than name [company]'s flagship if it's not something I actually live in.

A bit of why it's in my day. I lead a global engineering team across time zones, so I'm in collaboration, productivity and AI tools constantly. And in my work I've spent a lot of time sitting with underwriters to understand where their day breaks down, so I tend to look at every product the same way: where does the user's day get easier, and where does it still break?

What it does really well is **[VERIFY: the specific strength]**. That matters to someone like me because **[VERIFY: why it matters to users like you]**. When a product removes a step I'd otherwise do by hand and the care shows in the details, I notice it.

Where I'd improve it comes from a real moment: **[VERIFY: a specific moment it fell short for you or your team]**. So the one change I'd make is **[VERIFY: one focused improvement]**. I'd deliberately keep it to one focused change rather than a wish-list, because a list of features with no user need behind it isn't product thinking. I did consider **[VERIFY: a second idea]**, but this one addresses a more frequent need with less risk.

How I'd know it worked: I'd measure **[VERIFY: the metric, e.g., time-to-catch-up or missed mentions]** in a staged rollout or an A/B test, with a guardrail metric. And the trade-off I'd watch closely is **[VERIFY: e.g., noise vs completeness]**. Honestly, that's the hard part. Most improvements make something else slightly worse, and I'd want user research before building, because my own use is one data point, not a sample. It's the same approach I took with AI for our underwriters. The ask was "make underwriters faster," and the real need only became clear when we watched their day: they were re-keying data that already existed. We built around that, measured handling time, and kept a close eye on accuracy as the trade-off.

So for me, good product feedback starts from a real moment of use and ends with a metric and a trade-off, not just an opinion. And I'd genuinely like to hear what the team has already learned here, because I'm sure you've seen data I haven't.

B53. What motivates you beyond compensation and title? Where do you want to be in five years?

Also asked as: “What gets you out of bed?”; “What's your long-term career goal?”

What motivates me is building teams that build things that matter. And in five years, I want to be leading a larger engineering organization on platform or AI products used at large scale, known for developing leaders and for engineering quality.

I've been doing this for 17 years. I started as a software engineer, and today I'm a Director leading 80+ engineers. When I think about the moments that actually stick with me, they're not titles or org charts. They're **[VERIFY: engineers I hired becoming managers and tech leads]**, and they're underwriters doing in hours what used to take them days after we shipped intelligent document processing.

So the first thing that drives me is people growth. Honestly, the thing I'm proudest of isn't a system, it's **[VERIFY: the N engineers who became leaders]** on my teams. I invest heavily there: promotions into management, 90-day plans for new managers, regular coaching. The second is customer outcomes. I'm most energized when what we build visibly changes someone's day. With IDP I spent time with underwriters before and after, and seeing that difference is what keeps the team motivated too, not just me.

On the five-year piece, I've deliberately set a direction rather than a title. I could name a specific title, but that's not what drives me, and it wouldn't be honest. Where I want to be is leading a larger org of leaders, **[VERIFY: at director or senior director level]**, on platform or AI products at large scale, with a reputation for developing people and for engineering quality and reliability.

I'll be honest that it hasn't all gone well. Not every engineer I've invested in has grown into a leader. **[VERIFY: One manager I promoted struggled]**, and I learned from that to set much clearer success criteria up front. That's part of what I bring: I've made those mistakes at 80 people, and I'd rather not repeat them at a bigger scale. The way I measure it is by the leaders I've grown, the outcomes of what my teams ship, and whether my org runs well without me in every decision. This role is the right next step because **[VERIFY: the specific scope of this role]** moves me toward exactly that.

Earlier in my career I'd probably have said I wanted a title. I'd tell my younger self to focus on the people and the products sooner, because titles follow scope, and scope follows the leaders you grow.

11. Loop-specific and follow-on questions

F14. What's your biggest accomplishment? (Or: your most significant project)

Also asked as: “What are you proudest of?”, “What's the most impactful thing you've done?”, “If you could only tell us about one thing...”

Honestly, the thing I'm proudest of is changing how our specialty underwriting business actually gets work done. Over about **[VERIFY: ~3 years]**, we went from underwriters re-keying broker submissions into an old monolith to a modern platform with AI-assisted intake, and I built the 80-plus person org that runs it without me in every decision.

When I took it on, underwriters across Global Specialty and Small Specialty were typing submissions by hand into **[VERIFY: a monolithic on-prem underwriting system]** that released every **[VERIFY: two months]**. Submission-to-quote took **[VERIFY: days]**, and we were losing deals on turnaround. In a roughly $500 million business, that matters. Modernization and AI were both being talked about, but as two separate wish-lists with no funding.

So the first thing I did was make it one program with one number: quote-ready data within **[VERIFY: hours]** of a submission landing. The alternative was two programs with two sponsors, and I could see they'd end up fighting over the same engineers. Second, I put the platform first, the API layer and a document service, so intelligent document processing would plug into the new platform. The tempting move was an AI demo on the old system. It would've looked great for a quarter, and we'd have rebuilt it a year later. Third, I didn't ask for one big multi-year check. I asked for **[VERIFY: one phase]** at a time, with exit metrics agreed with underwriting leadership, and reported against the business number every quarter. And I organized teams around capabilities and grew managers from within, so platform, intake and AI each had an owner.

It wasn't smooth. Data migration was the long pole on every extraction, and dual-running cost more than we planned **[VERIFY: for ~18 months]**. My own instinct to build our own models on the AI side was wrong, and we had to change course. There were quarters where the business saw the cost before the benefit, and I had to keep earning the next phase.

Where it landed: turnaround went from **[VERIFY: days]** to **[VERIFY: hours for covered documents]**, releases went from **[VERIFY: every 2 months]** to **[VERIFY: weekly]**, and underwriter handling time dropped by **[VERIFY: about half]**, with zero business-stopping cutovers. The org went from about 20 engineers and one manager to 80-plus with **[VERIFY: N]** managers, **[VERIFY: M]** promoted from within.

So what I'm proudest of isn't any one launch. It's that the platform, the AI and the org were designed as one system, and it keeps improving without me. Now I start every big bet by asking which single business number it has to move.

F15. We see you at the level below what you applied for — why should we hire you at this level?

Also asked as: “Why do you think you're at this level?”, “We're thinking of you for the senior EM role rather than Director/M2 — thoughts?”, “What evidence do you have that you've operated at this scope?”

> That's fair to raise, and honestly I'd rather talk about it openly than guess. Titles don't map cleanly across industries. My title at The Hartford is Director, and I'd map that to a senior engineering manager or senior manager at most tech companies. So I'm not attached to the label. I'd just like to be measured against the scope I've actually run.
>
> Concretely: I lead 80-plus engineers through **[VERIFY: N]** managers, and I grew that from about 20. We support a roughly $500 million specialty insurance business across two lines, Global Specialty and Small Specialty. I own **[VERIFY: a budget of ~$X, including cloud and vendor spend]**, and I hire, develop and, when I have to, replace managers.
>
> The clearest example is the year I set our multi-year underwriting platform strategy. The business was adding products faster than a **[VERIFY: monolithic]** platform and a 20-person team could handle, there was no technical plan beyond the next project, and funding was decided one project at a time. I wrote a **[VERIFY: three-year]** direction, incremental modernization plus AI-assisted intake, and I was explicit about what we wouldn't do, mainly a big-bang rewrite pitched as one huge investment. I built the budget case with finance and business leadership and asked for funding phase by phase against exit metrics. And I designed the org to deliver it: capability teams, a leadership bench grown from within plus a few experienced hires, and written decision rights, instead of one flat team reporting to me. Nobody handed me the strategy, the money or the org design; I had to go and take ownership of them. Releases went from **[VERIFY: every 2 months]** to **[VERIFY: weekly]**, and the org grew to 80-plus with **[VERIFY: N]** managers.
>
> What would help me most is understanding where you saw the gap. Was it scope, the size of the org or the business? I'd be the first to say 80 engineers is smaller than many directors here, and I'd expect to grow into larger scope, not start there. Or was it a specific signal from the loop, like technical depth or how I talked about strategy? If it's something I didn't show well, I'd welcome the chance to address it with a specific example.
>
> And to be clear, I'm here because of the work and the team. If after that conversation you still see me a level below, I'm open to discussing it. I'd just want to understand the real scope of the role and what the path from there looks like. For me, level shows in what you own when nobody hands it to you: strategy, budget and leaders.

F16. What are you working on right now, what's most exciting — and what would you change about your current role?

Also asked as: “What's on your plate this quarter?”, “What do you enjoy most about your current job?”, “If you could change one thing about your role...”

Right now there are three things on my plate this **[VERIFY: quarter]**. First, we're extending our document-AI program from **[VERIFY: submissions to loss runs and schedules]**, which means new evaluation sets and new review workflows for underwriters. Second, we're **[VERIFY: retiring the last major piece of the legacy underwriting system]**. And third, I'm growing my next layer of leaders. **[VERIFY: Two]** of my managers are taking on bigger scope, and I'm coaching them through it.

The most exciting part, honestly, is the AI work. We've moved from "can the model read a document" to "can an underwriter trust it enough to change how they work," and that's as much measurement and workflow as it is models. Underwriting leadership came to us asking for something like ChatGPT for submissions. Extraction was already live and trusted for **[VERIFY: core submission documents]**, so the easy move would've been to let the team demo a general chatbot. I didn't want that. Instead I sat down with **[VERIFY: two underwriting leads]** and we defined what a useful summary contains and what counts as an unacceptable error, before anyone picked a model.

Then I asked the team for a golden set of **[VERIFY: ~200]** submissions with underwriter-written summaries, scored on factual accuracy and missing risk factors, rather than judging quality on demo impressions. Every summary cites its source page, and underwriters rate each one, so that feedback is our quality signal. The hard part is there's no single accuracy number for a summary. We're defining what "wrong" means with underwriters, case by case. We're at **[VERIFY: 15]** pilot underwriters and **[VERIFY: X%]** accuracy on the golden set, and watching them pull it into their day is the best part of my week. My takeaway so far: with generative AI, the evaluation set is the product spec.

What would I change? The Hartford has been a great place to build. The business partners are strong, they've backed big bets, and I've been able to change a lot: the org, the platform, how we fund work. What I can't change is the scale. Our problems are bounded by one insurer's volume and one industry's data, and engineering enables the product rather than being the product. I'd like more of my time on bigger technical problems where engineering is the product. That's a reason to move, not a complaint.

Which is a big part of why this role interests me: **[VERIFY: company-specific — the scale of the platform, the AI products, the engineering culture]**.

F17. How do you manage up? What kind of manager do you work best with?

Also asked as: “How do you work with your boss?”, “Describe your ideal manager.”, “How do you keep leadership informed?”, “What kind of manager brings out your best?”

For me, managing up is my job, not my manager's, and mostly it's about making their job easier. My first rule is no surprises: my manager never hears bad news about my org from someone else. I flag risks early, while there are still options, and I send a short written update every **[VERIFY: week]** covering what moved, what's at risk and what I need from them.

When I need a decision, I bring two or three options, the trade-offs, and my recommendation. That respects their time and makes it easy to say yes or redirect. I'll disagree in private, with data. But once a decision's made, I own it in front of my team as if it were mine.

The other piece is that I adapt to them, not the other way round. A good example: **[VERIFY: a new VP joined from finance/operations]** with a very different style from my previous boss. I'd been sending long narrative updates, and after about two weeks it was clear they weren't landing. A decision I needed on **[VERIFY: the next phase of the platform program]** was stalling. I could've escalated around them, or added more detail to win the argument. Instead I just asked: how do you like to get information, and what do you want to decide versus just know about?

Then I changed the format to a one-page weekly view: three metrics, top risks, and decisions needed with options. And we wrote down which calls were mine, which were theirs, and which needed the business. Honestly, it was a little uncomfortable admitting my updates weren't working, because I thought they were thorough. But time to decision on escalations went from **[VERIFY: weeks]** to **[VERIFY: days]**, and the stalled decision moved: **[VERIFY: phase funded]**. Now, in the first month with any new leader, I ask the same three questions (how do you like information, what do you want to decide versus know, and what keeps you up at night) and I write down decision rights.

As for the manager I work best with: someone who sets clear outcomes and context, gives me room on the how, and tells me directly when I'm off. I've worked well with very different styles. The one thing I find hard is ambiguity about decision rights. I've had **[VERIFY: a manager who left decision rights vague]**, and I learned to just ask, "is this mine to decide or yours?" rather than guess. So I clarify that early, and it saves us both a lot of friction.

F18. Tell me about your best hire — and a hire that didn't work out.

Also asked as: “What's the best hiring decision you've made?”, “Tell me about a mis-hire and what you learned.”, “How do you know when a hire isn't working?”

Sure, and actually both of them changed how my org hires, so they belong together. My best hire was **[VERIFY: an engineer from outside insurance]** I pushed for against a "needs domain experience" bar, and my worst was **[VERIFY: a senior hire]** who interviewed brilliantly and didn't work out.

While we were growing from about 20 to 80-plus, we were hiring **[VERIFY: ~15–20]** people a year across **[VERIFY: two countries]**. Our loop leaned on résumé, domain knowledge and a whiteboard, and honestly different interviewers were using different bars. I owned the hiring bar and the final call on senior hires.

On the best hire, the panel preferred an insurance-experienced candidate. I argued for **[VERIFY: a candidate from e-commerce with no insurance background]** because her work sample showed how she broke down an ambiguous problem. My view was that domain can be taught in months and judgment can't, and I took that risk on myself in the debrief. She became **[VERIFY: tech lead in ~1 year, manager in ~2]**, and she now leads **[VERIFY: our platform team]**.

The mis-hire was **[VERIFY: a senior engineer hired for a lead role]**. By about **[VERIFY: ~60 days]** I could see he stalled without precise requirements and was clashing with partner teams. I didn't wait for the annual review to see if he'd settle in. I gave him specific feedback with examples, and we wrote a 60-day plan together. When it wasn't met, I worked with HR and we **[VERIFY: parted ways / moved him to a narrower role]** at **[VERIFY: ~5 months]**. I explained the change to the team without private details and covered the gap. The hard part was admitting to the panel and my boss that I'd championed him **[VERIFY: if true]**.

What struck me was that both hires pointed at the same thing, so I fixed the loop, not just the case. We added a work sample on an ambiguous problem, a collaboration interview with an engineer from a partner team, written rubric feedback before the debrief, and structured reference questions on working with others. I deliberately didn't just add rounds; that slows hiring without better signal. The work sample did lengthen time to hire at first, until we trimmed it to **[VERIFY: 90 minutes]**. First-year mis-hires went from **[VERIFY: baseline]** to **[VERIFY: lower after loop change]**, and offer acceptance **[VERIFY: held steady]**.

The lesson for me is that a great interview is a weak predictor. How someone works on an ambiguous problem with other people is a strong one, so now we test for it on purpose.

F19. Tell me about a time you genuinely messed up a collaboration with another team.

Also asked as: “Tell me about a time you damaged a relationship with a partner team.”, “When did you let another team down?”, “Tell me about a cross-team failure that was your fault.”

Yes, and this one was clearly on me. On our document-AI program, I treated our privacy and information-security team as a late approval step, and then I described them as a blocker to leadership.

We were piloting intelligent document processing on broker submissions, which are full of policyholder personal and financial data. We'd designed a pipeline sending documents to **[VERIFY: a vendor's cloud extraction service]**, and I'd booked privacy review **[VERIFY: two weeks]** before go-live. That was my plan. When privacy looked at it, they flagged **[VERIFY: data retention and residency terms]** that meant redesigning the pipeline. They were right. But in my next leadership update, I framed the delay as their team blocking us.

Then I went to their lead and said plainly: we brought you in too late, and my update describing your team as a blocker was unfair. I didn't ask for an exception to launch, and I didn't argue the vendor terms were standard. Either one might have saved us a quarter and wrecked the relationship we'd need for every AI project after. Then I sent a correction to the same leadership audience saying the delay was our sequencing, not their review. Honestly, that was the hardest part, admitting publicly that my first update was wrong. My own team was frustrated by the delay too, and I had to own that with them.

After that I asked for a named privacy partner in our weekly design review, and we redesigned together: **[VERIFY: masking sensitive fields before extraction, contractual retention limits, encryption keys we controlled]**. And I made it a rule in my org that anything touching customer data gets a privacy and security design review at kickoff, before vendor selection.

It cost us **[VERIFY: ~6 weeks]** on the pilot. But privacy reviews on later AI projects went from **[VERIFY: weeks, late-stage]** to **[VERIFY: days, at kickoff]**, we've had **[VERIFY: 0]** late-stage privacy redesigns since, and the relationship turned around completely: **[VERIFY: privacy lead co-presented our approach to leadership]**.

Looking back, my mistake wasn't missing a step. It was thinking of a partner team as an obstacle to my schedule. Now I bring control functions in while the design is still cheap to change, and I never explain my delays with someone else's name.

F20. Tell me about a time budget pressure forced you to change direction mid-project.

Also asked as: “Tell me about a time a budget constraint changed your technical direction.”, “When did cost force you to rethink a plan?”, “How did you handle a mid-year budget cut?”

The one that comes to mind is our document-AI rollout. Midway through, our per-page vendor costs were on track to overrun budget by **[VERIFY: ~40%]**, and I ended up changing the pipeline design instead of the goal.

Adoption was faster than we'd planned, which was great, but submissions carried far more pages than our forecast assumed, with **[VERIFY: loss runs and schedules running to 100+ pages]**. Every page went through **[VERIFY: the vendor's premium extraction API]**. At the same time, the business asked every program to **[VERIFY: hold or reduce spend for the second half]**. I owned the budget and the outcome, so the job was to get back inside budget without giving up the underwriter time savings.

So the first thing I did was ask the team to break spend down per submission, by document type and page class. That's where it got interesting: **[VERIFY: ~60%]** of pages were boilerplate, blank, or forms we didn't even extract from. I could've gone to finance for more money on the strength of adoption, but it wasn't really available, and it would've hidden the problem. I also didn't want to stop rollout to new underwriters. That protects the budget but throws away the benefit.

Instead we changed the architecture. We added a cheap page-classification step and routed only pages with fields we needed to premium extraction, while structured forms went through **[VERIFY: standard OCR plus our own templates]**. With procurement I moved us to **[VERIFY: tiered volume pricing]**. And I paused expansion to **[VERIFY: one low-value document type]** for two quarters. Its business owner wasn't happy, and I had to own that trade-off with them directly. To make sure we weren't quietly degrading quality, we re-checked field accuracy on our golden set after every routing change. That mattered, because early on the classifier misrouted **[VERIFY: some schedules]** and accuracy dipped on one field until we tuned it.

Cost per submission went from **[VERIFY: $X]** to **[VERIFY: ~half]**, the projected overrun went from **[VERIFY: ~40% over]** to **[VERIFY: within budget]**, field accuracy was **[VERIFY: unchanged]**, and underwriters' handling-time savings were **[VERIFY: maintained]**. The paused document type **[VERIFY: resumed after 2 quarters]**.

What I took from it is that unit cost has to be a design input from the pilot, not a finance report after scale. We had the data and still forecast on averages. Now I won't approve an AI rollout without a cost-per-transaction model at production volumes.

F21. Our infrastructure spend has to drop 20% next quarter. What do you do?

Also asked as: “We need 20% off next quarter.”, “How would you cut cloud costs quickly?”, “Finance says your spend is too high — walk me through it.”

First, I'd treat it as a real constraint, not an opening bid. My goal would be 20% off the run-rate in a way that's predictable and doesn't trade away reliability. And I'd want finance as a partner, so I'd ask what's behind the number: a margin target, a forecast miss, or money moving somewhere else. That changes which levers make sense.

Week one is diagnosis. I'd get a cost view by team, service and environment, plus unit costs like cost per transaction. In my experience a few drivers explain most of it: idle non-production, overprovisioned compute and databases, storage on the wrong tier, and commitments we haven't bought. I've actually done a version of this. After our modernization, finance flagged that cloud spend had grown **[VERIFY: ~40%]** in two quarters against **[VERIFY: ~15%]** volume growth, with surprise bills hurting their forecast. The temptation was to defend it as the price of modernization. Instead we built that cost view together with finance and found exactly those drivers: always-on non-prod, oversized databases, and documents on hot storage long after 90 days.

Then I'd rank levers by savings, effort and risk. Fast and safe: schedule non-prod, delete orphans, tier storage, buy reserved capacity for steady load. Medium: rightsizing after load tests, autoscaling, tuning expensive queries. Slower and riskier: architecture changes or vendor renegotiation, which may land next quarter. I'd show finance that stack with dollar estimates and how confident I am in each. What I wouldn't do is a blanket 20% per team. It penalizes teams that are already efficient and pushes cuts into whatever's easiest to cut, which is usually headroom and observability.

Then I'd commit to a dated plan with an owner per lever, a weekly burn-down against the 20%, and a driver-based forecast so we can see if volume growth is eating the savings. If the safe levers only get us to **[VERIFY: 15%]**, I'd say so in week two with options for the gap, not in week twelve. And I'd tell the teams openly what the target is and why, because **[VERIFY: our best savings ideas came from the teams]**.

What I won't cut is production headroom for peaks, security controls, backups and DR, or the observability we need to run safely. In that earlier case we cut run-rate by **[VERIFY: ~30%]** while volume kept growing, saved about **[VERIFY: $X]** a year, and got the forecast **[VERIFY: within ±Y%]**. The real lesson was that finance doesn't need engineers to spend less so much as to spend predictably and explainably.

F22. How do you structure on-call so it stays sustainable?

Also asked as: “How do you run on-call?”, “How do you prevent on-call burnout?”, “What's a healthy pager load?”

I think on-call stays sustainable when the people carrying the pager have the power to make it quieter. So the team that builds a service runs it, and every page is either actionable or a bug we fix.

On design, I want rotations of at least **[VERIFY: six to eight]** people, so nobody's on more than about one week in six, with a primary and secondary and a written handoff at each change. With global teams, follow-the-sun where we can, so nobody's routinely paged at 3 a.m. New engineers shadow before they carry the pager, and every alert has a runbook. Then load: alerts on what users feel, which means SLO burn on latency and errors, not CPU. Each rotation has a page budget, around **[VERIFY: two off-hours pages a week]**. Above that, the top causes go into the next sprint ahead of features, and a weekly ops review looks at every page: was it actionable, what's the fix, who owns it.

I learned most of this the hard way. We had a quarter-end outage that users found before our alerts did. At the time we had **[VERIFY: rotations of 3–4 people]**, alerts on CPU and disk, and a few senior engineers carrying most of the incidents. The reflex after an outage is to add more alerts. I went the other way: teams rewrote alerting around SLOs and deleted alerts nobody acted on. We merged related services into larger shared rotations, set up **[VERIFY: follow-the-sun with our offshore team]**, and added shadowing.

The hard part was the page budget. It only works if leadership agrees in advance that it overrides the roadmap, so I negotiated that with our business partners in planning. Asking teams to fix noise in their spare time just doesn't happen.

On the people side, it's comp time after a bad night, no expectation of normal sprint output during an on-call week, and leaders who join major incidents so engineers aren't alone. Managers sit on an escalation rotation, not the primary pager. And I never praise heroics. I praise the fix that meant nobody had to be a hero. Over two quarters, off-hours pages dropped by **[VERIFY: ~60%]**, actionable pages went from **[VERIFY: ~40%]** to **[VERIFY: ~90%]**, and our SLO alerts started catching things before users did.

I track pages per shift, off-hours pages, time to restore, repeat incidents and a quarterly on-call survey. If those trend the wrong way, it's my job to fund the fix. Quiet on-call comes from investment, not toughness.

F23. What would you change about how [company] builds products — or AI?

Also asked as: “What would you improve about our product?”, “How would you change how we build AI?”, “What's one thing we're getting wrong?”

I'll caveat up front that I'm seeing this from the outside. You'll know trade-offs I don't, so take this as a hypothesis I'd want to test, not a verdict.

The way I think about any product change is pretty simple: whose problem is it, what evidence do I have, what's the hypothesis, what metric would prove it, and what's the risk if I'm wrong. That's how we ran every AI bet at The Hartford, and I'd use the same lens here.

So the one I'd pick is **[VERIFY: your pick — e.g., AI features that show their sources and confidence so users can verify before acting]**. The user problem I see is **[VERIFY: a specific user at a specific moment]**. My evidence is **[VERIFY: your own use, public reviews, or a customer's experience]**. My hypothesis is that if you **[VERIFY: the change]**, then **[VERIFY: the user behavior]** improves. Part of why I believe it is something we saw ourselves with document AI. When we started showing underwriters the model's confidence on each extracted field, trust and adoption went up, because people could see where to look instead of either checking everything or checking nothing.

How I'd test it: a small cohort first, with one primary metric, say task completion or retention for that user group, and a guardrail like error reports or support contacts. And I'd decide in advance what result would make us stop. Honestly, that's the part I've learned matters most, because the hardest thing is not falling in love with your own idea. On AI specifically, I'd want evaluation first: a golden set and a definition of failure before the demo, human feedback built into the product, and guardrail metrics for harm. That's what made our document AI something underwriters actually trusted.

The main risk is **[VERIFY: cost, latency, user confusion or privacy — name the one that fits your pick]**. And there are probably good reasons you haven't done it already that I can't see, whether that's cost, latency, regulation or a competing priority. My point is really the user problem, not my specific solution. If I were driving it, I'd start with the smallest experiment that tests the core hypothesis rather than asking for a big roadmap slot, and I'd staff it as a small cross-functional team: an engineering lead, product, design, and someone who owns evaluation and data, with a time box and a clear decision date.

I'd be curious how your team thinks about it, though. Is this something you've already explored?

F24. How do you balance team well-being against business needs? How do you prevent burnout?

Also asked as: “How do you avoid a death march?”, “How do you keep your team healthy during crunch?”, “Tell me about protecting your team from overwork.”

I honestly don't see it as a trade-off over any real time horizon. Burned-out teams ship slower, with more defects, and they lose their best people first. So for me, sustainable pace is a delivery decision, not a perk.

In practice, I watch leading signals, not just what people tell me: after-hours commits and pages, leave not taken, rising rework, missed one-on-ones, people going quiet in retros. My managers review those monthly, and in skip-levels I ask directly, "what's taking more energy than it should?" The levers are structural: cut or sequence scope, move a date with options, stop low-value work, fix the top sources of toil, rebalance on-call, add capacity. I'd rather have a hard conversation with a stakeholder about scope than ask people to absorb it quietly.

A real example: during our modernization, **[VERIFY: the same ~8 engineers]** on my platform team were supporting the legacy monolith and building the new services at the same time. Off-hours pages were up **[VERIFY: ~2×]**, two engineers hadn't taken leave in **[VERIFY: six months]**, and one told me in a skip-level she was "running on fumes." Honestly, that was on me. Dual-running is two jobs, and I'd staffed it as one.

The easy answer was to ask them to push through until legacy retirement. I didn't. I set up a rotating run squad, **[VERIFY: two engineers for a month at a time]**, owning legacy support so everyone else could build. I funded fixes for the top three causes of legacy pages and froze non-essential changes to the monolith. Then I went to the sponsor with the page and attrition-risk data, moved one extraction **[VERIFY: a quarter]** later and pulled a legacy retirement forward. I considered borrowing engineers from another team, but that would've just spread the problem. And both engineers took leave within the month, with cover arranged.

Off-hours pages went **[VERIFY: back to baseline]**, we had **[VERIFY: 0]** regretted attrition on the team, and apart from the one extraction we moved, the modernization milestones stayed **[VERIFY: on plan]**.

Sometimes the business genuinely needs a push, like a regulatory date or a peak period. When that happens I'm honest about it. It's time-boxed, I say what we're stopping to make room, leaders are in it with the team, and afterwards there's real recovery and a retro on why it happened, so crunch doesn't become the plan. And now I plan run and build capacity separately in any migration, and I track off-hours load as a program metric.

F25. How do you align with stakeholders while protecting your team?

Also asked as: “How do you shield your team from churn?”, “Tell me about a stakeholder demanding more than your team could deliver.”, “How do you handle mid-quarter asks?”

The way I think about it, protecting a team isn't refusing work. It's absorbing the ambiguity so they only see clear, stable decisions with the context behind them. A good example was when **[VERIFY: the head of a Small Specialty product line]** asked my manager to pull **[VERIFY: two senior engineers]** off our platform work mid-quarter.

She needed changes for **[VERIFY: a new product launch tied to a broker event]** in **[VERIFY: six weeks]**, and we were midway through **[VERIFY: a document-AI expansion milestone]**. The team had already absorbed **[VERIFY: two]** reprioritizations that year, and morale showed it. And my manager had half-agreed before I was involved, so I had to reset expectations without undermining him.

So the first thing I did was sit with her and her product lead and ask what had to be true on launch day. It turned out to be **[VERIFY: rating and form changes for one product]**, not the full workflow they'd asked for. Within 48 hours I gave her three options with costs: the launch minimum with one engineer for four weeks; the full ask with two engineers, slipping our milestone **[VERIFY: six weeks]**; or decline. I recommended the first. A flat no based on the quarterly plan would've protected the plan and lost the partner. A straight yes would've slipped our milestone and taught everyone that plans don't hold.

On the team side, I stayed the single interface to her team. I told my engineers what changed and why, because I don't believe in shielding people by keeping them in the dark. We picked a volunteer, and I time-boxed the loan in writing. That mattered, because the volunteer was worried about getting stuck on the other team. Everyone else's priorities stayed exactly the same.

Her launch **[VERIFY: met with the minimum scope]**, our platform milestone **[VERIFY: held within 2 weeks]**, and the relationship actually got stronger: **[VERIFY: she later sponsored our next phase]**. On the team, engagement **[VERIFY: recovered; no regretted exits]**. Then we made it a mechanism. Business leadership agreed that mid-quarter asks come with a named trade-off, and I commit to options within 48 hours.

Looking back, I'd have set up that intake rule before the first collision, not after the third, and briefed my manager earlier so asks reach me before they become commitments. What I've learned is that fast options earn more trust with stakeholders than either yes or no.

F26. Databricks panel Q&A — the seven questions the panel will likely ask after your presentation

Also asked as: Panel of directors and staff engineers after your ~25-minute presentation on the underwriting modernization with IDP (S2 + S3)

The panel will ask several of these after your presentation. Each answer is about 45 seconds.

“How did you keep data consistent while the legacy and new systems were running side by side?”

We kept the legacy system as the system of record until each cutover, and synced to the new services through **[VERIFY: change data capture on the legacy database into an event stream]**. Every event carried a business key and a version, and consumers were idempotent. They upserted by key and ignored older versions, so duplicates and replays were safe. On top of that, nightly reconciliation jobs compared **[VERIFY: policy, premium and status fields]** across both systems and opened a ticket for any mismatch. We only cut over a line of business after **[VERIFY: two weeks]** of zero unexplained differences. The trade-off was cost, because we dual-ran for **[VERIFY: ~18 months]**. If I did it again, I'd build that reconciliation tooling in the first quarter, not later.

“How did you measure model quality, and how did you catch drift?”

We built a golden dataset of **[VERIFY: ~1,000]** labeled documents per type, stratified by broker and scan quality, and measured precision and recall per field. We didn't use one overall accuracy number, because a wrong limit matters a lot more than a wrong fax number. In production, every underwriter correction becomes a label. For drift, we watched **[VERIFY: weekly]** field-level correction rates, confidence distributions and the input mix, like new brokers and new form versions, with alerts on shifts. When **[VERIFY: a large broker changed its form]**, correction rates spiked. We caught it in **[VERIFY: days]**, added samples and re-evaluated before redeploying. One gap we did find was that **[VERIFY: the golden set had too few low-quality scans]**, so we added stratification for that.

“Why did you set the confidence threshold where you did?”

We set it per field, not globally. The team took the precision–recall curve on the golden set for each field and picked the point that met a business rule underwriting leadership agreed to: errors reaching an underwriting decision stay below **[VERIFY: X%]**. High-impact fields like limits and premium got a higher threshold, so more of them go to human review. That costs reviewer time, and we accepted that. So really, it was a business risk decision expressed technically. I owned getting the rule agreed with underwriting, and the team owned the curves. It isn't fixed, either. We revisit it **[VERIFY: quarterly]** as accuracy improves, which is how auto-accept rates go up over time.

“What would you do differently with today's tools?”

Three things. First, I'd evaluate multimodal LLMs for the long tail of unstructured documents, but behind the same evaluation harness and confidence routing we already have, with cost and latency per document as first-class metrics, not afterthoughts. Second,

I'd put the document data, labels and features on one governed platform, **[VERIFY: lakehouse with lineage and access controls]**, instead of **[VERIFY: separate stores]**. That would make training data, lineage and access control much simpler. Third, I'd use managed CDC and data-quality tooling instead of the reconciliation code we wrote ourselves. Honestly, a lot of what we hand-built in that layer, I wouldn't build today. The core ideas, evaluation first and routing by confidence, I'd keep exactly as they are.

“What was the hardest people decision in the program?”

Moving a manager I'd promoted back to an engineering role. During the migration his team's attrition was **[VERIFY: about double]** our average, and honestly, I waited too long out of loyalty. Once I faced it, I gave him specific feedback, a coach and written criteria, with a fair three-month plan. When the criteria weren't met, I acted. I stepped in as interim manager to cover the team, and I hired his replacement with the team in the loop, so they had a say in who led them. What I changed afterwards is that every new manager in my org now gets written 90-day criteria, so neither of us is guessing whether it's working.

“How would the architecture hold up at 10× volume?”

Ingestion and extraction are stateless, so they scale horizontally behind a queue, and that part's fine. The constraints move elsewhere. First, vendor cost and rate limits. At 10× I'd tier routing harder and evaluate self-hosted models for the high-volume document types. Second, human review, which is the one people miss. At today's review rate, 10× volume means 10× reviewers, so raising auto-accept rates per field becomes the main lever. Third, the event and reconciliation pipeline, which needs partitioning by line of business and proper backpressure. And I'd load test against the real peak shape, which for us is quarter-end, not averages, because averages hide exactly the load that breaks you.

“Which decisions were yours, and which were your architects'?”

I decided the strategy and the bets. That meant incremental modernization over a rewrite, and buying extraction while building the specialty parts. That second one came after I'd been wrong about building our own first. I also decided the business metric we steered by, the sequencing by line of business, the threshold policy with underwriting, and go or no-go on every cutover. My architects and staff engineers designed the services, the CDC and event model, and the evaluation harness. I challenged their designs in review, sometimes hard, but they owned them. Happy to go deeper on any of those. For the finest implementation details, I'd tell you who owned it and how I'd find out.

F27. What questions do you have for us?

Also asked as: “Anything you'd like to ask me?”, end of every round

> Yes, thank you, I do. I've got a few, and I'll keep an eye on time.
>
> The first one: if we're sitting here a year from now and this hire has clearly worked out, what's different? And alongside that, what's the hardest problem I'd be inheriting on day one, the thing you'd want me to get my arms around first?
>
> Second, I'd love to understand decision rights. Which calls would be mine, and which would you want to keep with you? I ask because in my current role, getting that clear early has made more difference than almost anything else. It saves a lot of guessing on both sides.
>
> Third, something I've been thinking about since earlier in the loop, when **[VERIFY: tie to something you heard in an earlier round]**. What's the biggest technical bet this team is making right now, and what would make you reverse it?
>
> And the last one, if we have a couple of minutes: what separates the leaders who really thrive here from the ones who struggle in their first year? I'm more interested in the behaviors than the values on the wall.
>
> That's really helpful. You mentioned a moment ago that it comes down to how people handle ambiguity. Could you give me an example of what that looked like recently?
>
> Thanks. I've really enjoyed this conversation, and it's given me a much clearer picture of the team.

Swap in: For the hiring manager, ask "Where's the biggest gap on the current leadership team?" or "How much platform or tech-debt capacity is protected in planning?" For peers, ask "Where does this team's work most often collide with yours, and how do you resolve it?" or "What do you wish this team did differently?" For a skip-level, ask "How does this org's strategy connect to the company's top priorities this year?" For a Bar Raiser, AA or exec, ask "What's a recent decision the company got wrong, and how did it change course?" With an Amazon Bar Raiser, try "Which Leadership Principle do leaders in this org most often struggle with?" In a Microsoft AA round, try "What would you want me to have learned by the end of my first year here?" Keep compensation and level questions for the recruiter. In the follow-up line, swap in whatever the interviewer actually said.

12. More reported questions

B54. How do you manage people at very different seniority levels — e.g., a very senior engineer and a new grad, or a principal engineer who's senior to you technically?

Also asked as: “How do you manage someone more technical than you?”, “How does your approach change for juniors versus principals?”

Same bar, different support. With someone like a principal engineer, I bring context and criteria and give them real ownership. With early-career engineers, I bring structure, pairing and fast feedback.

A real example. **[VERIFY: Our principal architect]** knew far more about distributed systems than I did, and he wanted event streaming across all of our underwriting integrations. At the same time, as the org grew past **[VERIFY: 50]** engineers, we were hiring **[VERIFY: early-career engineers]** for the first time, and they needed much more structure than our senior folks. My job was to get the best out of both: use the principal's judgment without deferring blindly or overruling him, and get the new engineers productive and growing.

With the principal, I didn't try to compete on technology. I'd lose that, and it's not my job. Instead I brought what he didn't have, business context and operational risk, and we agreed decision criteria together first. Then I asked for a two-week spike so data decided. The easy option was to defer because he knew more, but that would have put streaming everywhere before we could actually run it. We landed on streaming for asynchronous, high-volume flows only, and then I gave him real ownership. He led the platform and wrote the standard, and I measured him on technical leverage across the org, not tickets.

Honestly, it was tense at first. He read my questions as doubting his expertise, and it was the agreed criteria that fixed that. It made it about the decision, not about him or me.

With the early-career engineers I did almost the opposite: a documented onboarding path, a named buddy, a small production change in week one, and weekly one-on-ones with written expectations. Treating them like seniors and letting them find their way works for a few and sinks most. Our seniors resisted buddy time until I counted it as part of their own performance. Then I connected the two groups. I asked the principal to mentor **[VERIFY: two]** early-career engineers and run a design clinic, which scaled his expertise. Streaming went from an all-or-nothing debate to **[VERIFY: 4 flows in 2 quarters with 0 streaming incidents]**, time to a new hire's first production change dropped from **[VERIFY: ~3 months]** to **[VERIFY: ~2 weeks]**, and **[VERIFY: N of N]** stayed past their first year. The principal now brings disagreements to me early.

What I've learned is that with someone more expert than me, my job is to improve the quality of the decision, not to win the argument. With juniors, it's to shorten the feedback loop.

B55. Tell me about driving an initiative through to launch end to end

Also asked as: “Tell me about a feature you drove through”, “Walk me through taking something from idea to production.”

The one I'd pick is taking a pretty vague executive ask, "use AI to make underwriters faster," all the way to a launched intelligent document processing capability that cut submission handling time **[VERIFY: by about half]**, with **[VERIFY: ~80%]** adoption.

The problem was that underwriters were re-keying broker submissions, things like applications, loss runs and schedules, before doing any actual underwriting. Each one took **[VERIFY: X hours]** of data entry, and in peak periods submissions waited **[VERIFY: days]**. In a ~$500M specialty book, we were losing business on turnaround. I owned turning that ask into a funded, measurable program and driving it through launch and adoption across underwriting, operations, privacy and my engineering teams.

So the first thing I did was reframe it. I spent a week with underwriters and defined the goal as structured, trustworthy data within an hour of a submission arriving, measured by handling time and by errors reaching underwriting decisions. The alternative was a broad "AI for underwriting" program with no clear metric, and I didn't want that. Second, I de-risked it with a pilot, six weeks, one document type, ten underwriters, exit criteria agreed with the business sponsor before we started, rather than funding the whole thing up front. Third, I made the build-buy call on evidence. My own instinct was to build our models end to end, and honestly, that was wrong. A benchmark showed bought extraction beat ours for standard fields, so we bought that and built the specialty classification, validation and review workflow.

Then I cleared the path. I ran a weekly cross-functional review with underwriting, operations and privacy, and we rolled out line by line with underwriter champions. What went wrong: the privacy review became the critical path because we'd brought them in too late, and one document type had scan quality too poor to extract, so we cut it from launch.

Handling time went from **[VERIFY: X hours]** to **[VERIFY: Y minutes]** for covered documents, about **[VERIFY: Z%]** of fields were accepted without correction, adoption went from a **[VERIFY: pilot of 10]** to **[VERIFY: ~80%]** of target underwriters, and quote turnaround improved **[VERIFY: by X%]**. Changing my mind on build versus buy got us live **[VERIFY: about a quarter earlier]**.

What I took away is that launch is the midpoint, not the finish. Now I define adoption and guardrail metrics before the pilot, bring privacy and risk in during week one, and stay on the weekly adoption review until usage is actually stable.

B56. Tell me about resolving a large-scale conflict — between teams or org-wide, not two individuals

Also asked as: “Tell me about resolving a conflict between two groups”, “How have you handled competing priorities across organizations?”

The one that comes to mind is a long-running conflict between our Global Specialty and Small Specialty engineering groups. I resolved it by treating it as a structural problem rather than a people problem, and we reorganized around shared capabilities with **[VERIFY: no regretted attrition]**.

[VERIFY: The two groups had grown separately, each with its own platform for intake and rating.] Each side's business leaders pushed their own priorities, any change to a shared component turned into a turf fight, and escalations were landing on me **[VERIFY: weekly]**. Meanwhile the business strategy was moving toward shared underwriting capabilities, so the structure was fighting the strategy. I owned resolving it across **[VERIFY: ~40]** engineers and two sets of business stakeholders, in a way that wouldn't just recreate the fight.

So the first thing I did was look at the data rather than the personalities. I mapped every escalation from the previous quarter, and **[VERIFY: most]** traced back to duplicated platforms and unclear ownership of shared components, not to individuals. I'd been mediating case by case, and that just kept me as the bottleneck. Second, I got both sides looking at the same picture, with a joint session for both managers and business leads on what each line needed and where handoffs were slowing both of them down. Third, I designed two options with my managers and then made the call: combined platform teams, product-aligned teams for each line, and written decision rights. Platform owns shared components, the lines own product priorities, and there's a joint quarterly backlog. I passed on a priority-arbitration committee because it would have institutionalized the fight.

The people side was the hardest. One business leader feared his line would lose priority, so I gave each line a guaranteed share of the joint backlog. Two managers' scopes shrank, and one took **[VERIFY: a couple of months]** to settle. Everyone whose scope changed heard it from me individually before any announcement.

Duplicated services went from **[VERIFY: X]** to **[VERIFY: Y]**, escalations dropped from **[VERIFY: about weekly]** to **[VERIFY: about monthly or less]**, cross-line feature lead time came down **[VERIFY: by X%]**, and regretted attrition through the change was **[VERIFY: zero]**. The decision rights are written down and used in quarterly planning.

What I learned is that when smart people keep fighting, look at the structure before the personalities. Looking back, the escalation data was there a few quarters earlier, and I'd act sooner.

B57. Tell me about a decision where you reasoned from first principles or where data contradicted the consensus

Also asked as: “When did the data tell you something everyone else disagreed with?”, “Tell me about a time you went deep to find the real answer.”

The clearest example is our cloud bill. Everyone agreed it was rising because of modernization, basically the price of being in the cloud, and when I broke it down to cost per submission, **[VERIFY: most]** of the growth had nothing to do with volume. We ended up cutting run-rate **[VERIFY: ~30%]** while volume kept growing.

After our modernization, cloud spend grew **[VERIFY: ~40% in two quarters]** while transactions grew **[VERIFY: ~15%]**. The shared view among my managers and architects was that this was expected, and **[VERIFY: a budget increase request]** was already being drafted. I owned the platform's cost, and before asking for more money, I wanted to know what we were actually paying for.

So I went back to the unit. The question I asked was simple: what should one submission cost to process? With finance, we built a view of cost per submission and per environment, by team and by service. The easy path was to accept the total and ask for the bigger budget, and that's where we were heading. The breakdown told a different story. The growth was coming from always-on non-production environments, oversized databases, and documents sitting on hot storage after 90 days. Mostly nothing to do with volume.

Then I tested it before acting. We switched off schedules for one team's non-production environments for two weeks and rightsized one database after a load test. Both held, with no performance impact. I deliberately didn't do a blanket percentage cut per team, because that punishes the teams that were already efficient.

The hard part was people, not numbers. Some of my architects felt the analysis was questioning their designs. I framed it as defaults nobody had revisited, not mistakes, which was true. And our tagging was incomplete, so **[VERIFY: ~20%]** of spend had to be attributed by hand. Monthly run-rate came down **[VERIFY: ~30%]**, cost per submission dropped **[VERIFY: X%]**, the budget increase request was withdrawn, and forecast accuracy got to **[VERIFY: within ±Y%]**. More importantly, it changed how we decide: a driver-based forecast reviewed monthly, and every team now sees and owns its unit cost.

What I learned is that totals hide causes. When a number moves now, I go to the unit, per submission, per user, per request, before accepting any explanation, including my own. And I'd build that unit-cost visibility in from the start of any cloud move.

B58. Tell me about inheriting a struggling team and turning it around

Also asked as: “You take over a team that's missing every commitment — what do you do?”, “Tell me about a team turnaround.”

Sure. When I inherited **[VERIFY: our rating integration team]**, it was missing most of its commitments and losing people. Within **[VERIFY: two quarters]**, it was meeting commitments, incidents were down sharply, and the business was routing work back to it.

[VERIFY: As the org grew, the team, about 10 engineers, moved under me.] It had met **[VERIFY: ~40%]** of commitments for three quarters, caused **[VERIFY: a large share]** of our production incidents, lost **[VERIFY: three]** engineers in six months, and underwriting leaders had started **[VERIFY: asking other teams to do its work]**. That last one is the real signal that trust is gone. My job was to turn it around, delivery, reliability, morale and stakeholder trust, without just taking over the work myself.

So the first thing I did was listen and measure before changing anything. In the first 30 days I met every engineer and the key stakeholders and pulled the delivery and incident data. The temptation is a fast reset in week one, new process and new priorities, but I didn't know the cause yet. The data pointed to three things: **[VERIFY: too many parallel requests from two lines]**, no test automation on a fragile integration, and a manager stretched across two teams. Not weak engineers. Overload and a leadership gap.

Then I made three changes. I agreed with the business leads to stop **[VERIFY: about a third]** of in-flight requests and run one intake queue, so the team could actually finish things. I gave the team a dedicated manager **[VERIFY: hired or promoted]** and a named tech lead, and wrote down what the team owned, rather than keeping the shared manager and bolting process around him. And we borrowed reliability practices from our strongest team: CI tests on the integration, feature flags, a postmortem for every incident, and capacity set aside for a quarter.

It wasn't easy. The business leads pushed back hard on stopping a third of their requests, and I had to show them the completion data. And one senior engineer was close to leaving. I nearly lost him before the changes started to show. Commitments met went from **[VERIFY: ~40%]** to **[VERIFY: ~85%]**, incidents per quarter dropped **[VERIFY: X%]**, we had **[VERIFY: zero regretted attrition in the next two quarters]**, and **[VERIFY: a new product integration was assigned back to the team]**.

What I took from it is that struggling teams are usually overloaded and under-led, not weak. Now I start every inheritance with 30 days of listening and data, change no more than three things, and make the first visible win about finishing, not starting.

B59. Tell me about a time you delegated something important and gave your team ownership

Also asked as: “How do you lead with context, not control?”, “Tell me about trusting your team with a decision you'd normally make.”

The one I'd pick is the rebuild of our document extraction pipeline, which was the core of our applied-AI program. I delegated it to a tech lead with the outcome, the guardrails and her decision rights in writing. She delivered it, and later **[VERIFY: became a manager]**.

Inside our intelligent document processing program, **[VERIFY: the extraction pipeline had to be rebuilt to handle the bought extraction service plus our specialty classification]**. It was high-visibility and on the critical path for underwriting adoption. And honestly, as someone who came up through engineering, my instinct was to stay close to the design. I owned the program's outcome with the business, but I chose to give a senior engineer real ownership of the rebuild: the design, the plan and the calls.

So the first thing I did was give context, not instructions. I wrote one page covering the outcome, data within an hour with an error guardrail; the constraints, privacy rules, budget and the launch window; and which decisions were hers versus mine. Only one-way doors like vendor contracts came to me. The option I passed on was keeping design approval myself, which would have made me the bottleneck and told her I didn't trust her. Second, we set up a fortnightly 30-minute review on outcome metrics and risks, with a standing rule: bring me bad news early and I'll help, not take over. Third, I made the ownership visible. She presented the design at architecture review and the results to **[VERIFY: business leadership]**, not me.

The hard part was mine. When she chose a **[VERIFY: queue-based retry design]** that I'd have done differently, I asked questions but let it stand because it was reversible. That was uncomfortable, and it worked. And **[VERIFY: once, around a vendor delay]**, she held bad news about a week too long. We reset the "bring it early" rule together, and it didn't happen again.

The rebuild was **[VERIFY: delivered on schedule]**, only **[VERIFY: 2]** decisions escalated to me, both one-way doors, and my time on it dropped to **[VERIFY: about 2 hours a fortnight]**. She grew from senior engineer to **[VERIFY: tech lead, then engineering manager]**. And it became how my org runs: every tech lead on a major initiative now gets written decision rights.

What I learned is that delegation fails when it's vague. People either ask permission for everything or get overruled late. Writing down the outcome, the guardrails and who decides what is what made it real. Next time I'd write the decision rights before choosing the person.